

Physlib Monthly Report

1–28 February 2026

Contributors: Curly-Howard-Chungus Correspondence |
Lamport-Cabot-Codd-Backus-Naur Form, Daniel Morrison, Gregory Loges,
Joseph Tooby-Smith, Nicola Bernini, Nikolai Kashcheev, Pietro Monticone, Zhi
Kai Pong

Reviewers: Daniel Morrison, Joseph Tooby-Smith, Zhi Kai Pong

Generated 14 August 2026 from commit 6286c6b.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–28 February 2026 on branch `master` of [leanprover-community/physlib](#), between commits `d127ae0` and `6286c6b`. Over that period 8 contributors landed 40 commits, changing 5,775 lines across 146 files and adding 323 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>PhysLean.ClassicalMechanics.DampedHarmonicOscillator.Basic</code>	3
2.2	<code>PhysLean.ClassicalMechanics.HarmonicOscillator.Basic</code>	3
2.3	<code>PhysLean.ClassicalMechanics.HarmonicOscillator.ConfigurationSpace</code> . .	4
2.4	<code>PhysLean.ClassicalMechanics.Mass.MassUnit</code>	8
2.5	<code>PhysLean.Electromagnetism.Charge.ChargeUnit</code>	8
2.6	<code>PhysLean.Mathematics.InnerProductSpace.Submodule</code>	9
2.7	<code>PhysLean.Mathematics.KroneckerDelta</code>	10
2.8	<code>PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.PlusU1.QuadSol</code> 10	
2.9	<code>PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.PlusU1.QuadSolToSol</code> 1	
2.10	<code>PhysLean.Particles.BeyondTheStandardModel.TwoHDM.GramMatrix</code>	10
2.11	<code>PhysLean.Particles.BeyondTheStandardModel.TwoHDM.Potential</code>	11
2.12	<code>PhysLean.Particles.FlavorPhysics.CKMMatrix.Relations</code>	15
2.13	<code>PhysLean.Particles.FlavorPhysics.CKMMatrix.StandardParameterization.StandardParameters</code>	
2.14	<code>PhysLean.Particles.StandardModel.AnomalyCancellation.NoGrav.One.Lemmas</code> 16	
2.15	<code>PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimalSuperSet</code> .	16
2.16	<code>PhysLean.QFT.PerturbationTheory.CreateAnnihilate</code>	16
2.17	<code>PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommute</code> . . .	17

2.18	PhysLean.QFT.PerturbationTheory.FieldSpecification.NormalOrder	17
2.19	PhysLean.QFT.PerturbationTheory.FieldSpecification.TimeOrder	17
2.20	PhysLean.QFT.PerturbationTheory.FieldStatistics.Basic	17
2.21	PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeContraction	18
2.22	PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeOrder	18
2.23	PhysLean.QFT.PerturbationTheory.WickContraction.Basic	18
2.24	PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContractNat	19
2.25	PhysLean.QFT.PerturbationTheory.WickContraction.Join	19
2.26	PhysLean.QFT.PerturbationTheory.WickContraction.Singleton	19
2.27	PhysLean.QFT.PerturbationTheory.WickContraction.UncontractedList . . .	20
2.28	PhysLean.QuantumMechanics.DDimensions.Hydrogen.Basic	20
2.29	PhysLean.QuantumMechanics.DDimensions.Hydrogen.LaplaceRungeLenzVector	20
2.30	PhysLean.QuantumMechanics.DDimensions.Operators.AngularMomentum . . .	26
2.31	PhysLean.QuantumMechanics.DDimensions.Operators.Commutation	27
2.32	PhysLean.QuantumMechanics.DDimensions.Operators.Momentum	30
2.33	PhysLean.QuantumMechanics.DDimensions.Operators.Position	31
2.34	PhysLean.QuantumMechanics.DDimensions.Operators.Unbounded	32
2.35	PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Basic	35
2.36	PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmodule	37
2.37	PhysLean.Relativity.Tensors.Contraction.Pure	38
2.38	PhysLean.Relativity.Tensors.Evaluation	39
2.39	PhysLean.Relativity.Tensors.RealTensor.ToComplex	39
2.40	PhysLean.Relativity.Tensors.RealTensor.Velocity.Basic	41
2.41	PhysLean.Relativity.Tensors.TensorSpecies.Basic	42
2.42	PhysLean.SpaceAndTime.Space.Basic	42
2.43	PhysLean.SpaceAndTime.Space.Derivatives.Basic	42
2.44	PhysLean.SpaceAndTime.Space.LengthUnit	42
2.45	PhysLean.SpaceAndTime.Time.TimeUnit	43
2.46	PhysLean.StatisticalMechanics.BoltzmannConstant	43
2.47	PhysLean.Thermodynamics.Temperature.TemperatureUnits	43
2.48	PhysLean.Units.Basic	43

1 Introduction

During Feb 2026, 8 contributors submitted 40 commits that changed 5,775 lines (+4,686 / -1,089) across 146 files spanning 22 top-level areas of the repository. Of those, 13 files were newly added. This report catalogues 323 newly added Lean declarations: 254 lemmas, 36 defs, 28 instances, 3 structures, and 2 abbrevs. We're delighted to recognize the following first-time contributors this month: [Curly-Howard-Chungus Correspondence](#) | [Lamport-Cabot-Codd-Backus-Naur Form](#) and [Gregory Loges](#).

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Curly-Howard-Chungus Correspondence Lamport-Cabot-Codd-Backus-Naur Form	342	1
Daniel Morrison	9	1
Gregory Loges	2,812	1
Joseph Tooby-Smith	839	1
Nicola Bernini	690	1
Nikolai Kashcheev	612	1
Pietro Monticone	127	1
Zhi Kai Pong	1,106	1

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Daniel Morrison	3
Joseph Tooby-Smith	14
Zhi Kai Pong	1

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
PhysLean/QuantumMechanics	15	+9	—	+2,009	-19
PhysLean/Particles	16	—	—	+711	-154
PhysLean/Relativity	19	—	—	+711	-145
PhysLean/ClassicalMechanics	7	+1	—	+532	-230
PhysLean/Mathematics	14	+2	—	+339	-202
PhysLean/QFT	39	—	—	+176	-183
PhysLean/SpaceAndTime	14	—	—	+107	-95
lake-manifest.json	1	—	—	+16	-16
docs	1	+1	—	+26	-0
PhysLean/Electromagnetism	4	—	—	+12	-12
PhysLean/Thermodynamics	2	—	—	+9	-9

PhysLean.lean	1	–	–	+12	-0
.github/ISSUE_TEMPLATE	1	–	–	+7	-4
PhysLean/StatisticalMechanics	3	–	–	+5	-5
PhysLean/CondensedMatter	1	–	–	+5	-4
PhysLean/Units	2	–	–	+3	-3
lakefile.toml	1	–	–	+2	-2
PhysLean/Meta	1	–	–	+1	-2
README.md	1	–	–	+1	-1
docs/_data	1	–	–	+1	-1
lean-toolchain	1	–	–	+1	-1
scripts/MetaPrograms	1	–	–	+0	-1

1.4 New declarations by kind

Kind	Count
lemma	254
def	36
instance	28
structure	3
abbrev	2

2 Details by file

2.1 PhysLean.ClassicalMechanics.DampedHarmonicOscillator.Basic

[PhysLean/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean](#) on GitHub.

lemma k_ne_zero

[\[source\]](#)

```
@[simp]
lemma k_ne_zero : S.k ≠ 0
```

lemma m_ne_zero

[\[source\]](#)

```
@[simp]
lemma m_ne_zero : S.m ≠ 0
```

2.2 PhysLean.ClassicalMechanics.HarmonicOscillator.Basic

[PhysLean/ClassicalMechanics/HarmonicOscillator/Basic.lean](#) on GitHub.

lemma k_ne_zero

[\[source\]](#)

```
@[simp]
lemma k_ne_zero : S.k ≠ 0
```

lemma m_ne_zero

[\[source\]](#)

```
@[simp]
lemma m_ne_zero : S.m ≠ 0
```

lemma ω_{ne_zero} [\[source\]](#)

The angular frequency of the classical harmonic oscillator is not equal to zero.

```
lemma  $\omega_{ne\_zero}$  : S. $\omega \neq 0$ 
```

lemma toDual_symm_innerSL[\[source\]](#)

```
lemma toDual_symm_innerSL (x : ConfigurationSpace) :  
  (InnerProductSpace.toDual  $\mathbb{R}$  ConfigurationSpace).symm (innerSL  $\mathbb{R}$  x) = x
```

lemma gradient_inner_self[\[source\]](#)

```
lemma gradient_inner_self (x : ConfigurationSpace) :  
  gradient (fun y : ConfigurationSpace =>  $\langle\langle y, y \rangle\rangle_{\mathbb{R}}$ ) x = (2 :  $\mathbb{R}$ ) • x
```

lemma gradient_const_mul_inner_self[\[source\]](#)

```
lemma gradient_const_mul_inner_self (c :  $\mathbb{R}$ ) (x : ConfigurationSpace) :  
  gradient (fun y : ConfigurationSpace => c *  $\langle\langle y, y \rangle\rangle_{\mathbb{R}}$ ) x = (2 * c) • x
```

lemma gradient_add_const'[\[source\]](#)

```
private lemma gradient_add_const' {f : ConfigurationSpace →  $\mathbb{R}$ } {c :  $\mathbb{R}$ }  
  (x : ConfigurationSpace) :  
  gradient (fun y => f y + c) x = gradient f x
```

2.3 PhysLean.ClassicalMechanics.HarmonicOscillator.ConfigurationSpace

[PhysLean/ClassicalMechanics/HarmonicOscillator/ConfigurationSpace.lean](#) on GitHub.

structure ConfigurationSpace[\[source\]](#)

The configuration space of the harmonic oscillator.

```
structure ConfigurationSpace where  
  val :  $\mathbb{R}$ 
```

lemma ext[\[source\]](#)

```
@[ext]  
lemma ext {x y : ConfigurationSpace} (h : x.val = y.val) : x = y
```

instance Zero ConfigurationSpace[\[source\]](#)

```
instance : Zero ConfigurationSpace
```

instance OfNat ConfigurationSpace 0[\[source\]](#)

```
instance : OfNat ConfigurationSpace 0
```

lemma zero_val[\[source\]](#)

```
@[simp]
lemma zero_val : (0 : ConfigurationSpace).val = 0
```

instance Add ConfigurationSpace[\[source\]](#)

```
instance : Add ConfigurationSpace where
```

lemma add_val[\[source\]](#)

```
@[simp]
lemma add_val (x y : ConfigurationSpace) : (x + y).val = x.val + y.val
```

instance Neg ConfigurationSpace[\[source\]](#)

```
instance : Neg ConfigurationSpace where
```

lemma neg_val[\[source\]](#)

```
@[simp]
lemma neg_val (x : ConfigurationSpace) : (-x).val = -x.val
```

instance Sub ConfigurationSpace[\[source\]](#)

```
instance : Sub ConfigurationSpace where
```

lemma sub_val[\[source\]](#)

```
@[simp]
lemma sub_val (x y : ConfigurationSpace) : (x - y).val = x.val - y.val
```

instance SMul $\mathbb{R}$ ConfigurationSpace[\[source\]](#)

```
instance : SMul  $\mathbb{R}$  ConfigurationSpace where
```

lemma smul_val[\[source\]](#)

```
@[simp]
lemma smul_val (r :  $\mathbb{R}$ ) (x : ConfigurationSpace) : (r • x).val = r * x.val
```

instance CoeFun ConfigurationSpace (fun _ => Fin 1 $\rightarrow$ $\mathbb{R}$)[\[source\]](#)

```
instance : CoeFun ConfigurationSpace (fun _ => Fin 1  $\rightarrow$   $\mathbb{R}$ ) where
```

lemma apply_zero[\[source\]](#)

```
@[simp]
lemma apply_zero (x : ConfigurationSpace) : x 0 = x.val
```

lemma apply_eq_val[\[source\]](#)`@[simp]``lemma apply_eq_val (x : ConfigurationSpace) (i : Fin 1) : x i = x.val`**instance AddGroup ConfigurationSpace**[\[source\]](#)`instance : AddGroup ConfigurationSpace where`**instance AddCommGroup ConfigurationSpace**[\[source\]](#)`instance : AddCommGroup ConfigurationSpace where`**instance Module  $\mathbb{R}$  ConfigurationSpace**[\[source\]](#)`instance : Module $\mathbb{R}$ ConfigurationSpace where`**instance Norm ConfigurationSpace**[\[source\]](#)`instance : Norm ConfigurationSpace where`**instance Dist ConfigurationSpace**[\[source\]](#)`instance : Dist ConfigurationSpace where`**lemma dist\_eq\_val**[\[source\]](#)`lemma dist_eq_val (x y : ConfigurationSpace) :
 dist x y = ||x.val - y.val||`**instance SeminormedAddCommGroup ConfigurationSpace**[\[source\]](#)`instance : SeminormedAddCommGroup ConfigurationSpace where`**instance NormedAddCommGroup ConfigurationSpace**[\[source\]](#)`instance : NormedAddCommGroup ConfigurationSpace where`**instance NormedSpace  $\mathbb{R}$  ConfigurationSpace**[\[source\]](#)`instance : NormedSpace $\mathbb{R}$ ConfigurationSpace where`**instance Inner  $\mathbb{R}$  ConfigurationSpace**[\[source\]](#)`instance : Inner $\mathbb{R}$ ConfigurationSpace where`**lemma inner\_def**[\[source\]](#)`@[simp]``lemma inner_def (x y : ConfigurationSpace) : $\langle\langle x, y \rangle\rangle_{\mathbb{R}} = x.val * y.val$`

instance InnerProductSpace $\mathbb{R}$ ConfigurationSpace

[\[source\]](#)

```
noncomputable instance : InnerProductSpace  $\mathbb{R}$  ConfigurationSpace where
```

lemma differentiable_inner_self

[\[source\]](#)

```
@[fun_prop]
lemma differentiable_inner_self :
  Differentiable  $\mathbb{R}$  (fun x : ConfigurationSpace =>  $\langle\langle x, x \rangle\rangle_{\mathbb{R}}$ )
```

lemma differentiableAt_inner_self

[\[source\]](#)

```
@[fun_prop]
lemma differentiableAt_inner_self (x : ConfigurationSpace) :
  DifferentiableAt  $\mathbb{R}$  (fun y : ConfigurationSpace =>  $\langle\langle y, y \rangle\rangle_{\mathbb{R}}$ ) x
```

lemma contDiff_inner_self

[\[source\]](#)

```
@[fun_prop]
lemma contDiff_inner_self (n : WithTop  $\mathbb{N}_{\infty}$ ) :
  ContDiff  $\mathbb{R}$  n (fun x : ConfigurationSpace =>  $\langle\langle x, x \rangle\rangle_{\mathbb{R}}$ )
```

def toRealLM

[\[source\]](#)

Linear map sending a configuration space element to its underlying real value.

```
noncomputable def toRealLM : ConfigurationSpace  $\rightarrow_l$   $[\mathbb{R}]$   $\mathbb{R}$ 
```

def fromRealLM

[\[source\]](#)

Linear map embedding a real value into the configuration space.

```
noncomputable def fromRealLM :  $\mathbb{R} \rightarrow_l$   $[\mathbb{R}]$  ConfigurationSpace
```

def toRealCLM

[\[source\]](#)

Continuous linear map sending a configuration space element to its underlying real value.

```
noncomputable def toRealCLM : ConfigurationSpace  $\rightarrow_L$   $[\mathbb{R}]$   $\mathbb{R}$ 
```

def fromRealCLM

[\[source\]](#)

Continuous linear map embedding a real value into the configuration space.

```
noncomputable def fromRealCLM :  $\mathbb{R} \rightarrow_L$   $[\mathbb{R}]$  ConfigurationSpace
```

def valHomeomorphism

[\[source\]](#)

Homeomorphism between configuration space and $\mathbb{R}$ given by ConfigurationSpace.val.

```
noncomputable def valHomeomorphism : ConfigurationSpace  $\approx_t$   $\mathbb{R}$  where
```


instance ChartedSpace $\mathbb{R}$ ConfigurationSpace [\[source\]](#)

The structure of a charted space on ConfigurationSpace.

noncomputable instance : ChartedSpace $\mathbb{R}$ ConfigurationSpace **where**

instance IsManifold $\mathcal{I}(\mathbb{R}, \mathbb{R}) \omega$ ConfigurationSpace [\[source\]](#)

The structure of a smooth manifold on ConfigurationSpace.

noncomputable instance : IsManifold $\mathcal{I}(\mathbb{R}, \mathbb{R}) \omega$ ConfigurationSpace **where**

instance FiniteDimensional $\mathbb{R}$ ConfigurationSpace [\[source\]](#)

instance : FiniteDimensional $\mathbb{R}$ ConfigurationSpace

instance CompleteSpace ConfigurationSpace [\[source\]](#)

instance : CompleteSpace ConfigurationSpace

def toSpace [\[source\]](#)

The position in one-dimensional space associated to the configuration.

def toSpace (q : ConfigurationSpace) : Space 1

lemma toSpace_apply [\[source\]](#)

@[simp]
lemma toSpace_apply (q : ConfigurationSpace) (i : Fin 1) : q.toSpace i = q.val

2.4 PhysLean.ClassicalMechanics.Mass.MassUnit

[PhysLean/ClassicalMechanics/Mass/MassUnit.lean](#) on GitHub.

lemma val_ne_zero [\[source\]](#)

@[simp]
lemma val_ne_zero (x : MassUnit) : x.val $\neq$ 0

lemma div_ne_zero [\[source\]](#)

@[simp]
lemma div_ne_zero (x y : MassUnit) : $\neg$ x / y = (0 : $\mathbb{R}_{\geq 0}$)

2.5 PhysLean.Electromagnetism.Charge.ChargeUnit

[PhysLean/Electromagnetism/Charge/ChargeUnit.lean](#) on GitHub.

lemma val_ne_zero[\[source\]](#)

```
@[simp]
lemma val_ne_zero (x : ChargeUnit) : x.val ≠ 0
```

lemma div_ne_zero[\[source\]](#)

```
@[simp]
lemma div_ne_zero (x y : ChargeUnit) : ¬ x / y = (0 : ℝ≥0)
```

2.6 PhysLean.Mathematics.InnerProductSpace.Submodule

[PhysLean/Mathematics/InnerProductSpace/Submodule.lean](#) on GitHub.

def submoduleToLp[\[source\]](#)

The submodule of $\text{WithLp } 2 \text{ } (E \times E)$ defined by M .

```
def submoduleToLp : Submodule ℂ (WithLp 2 (E × E)) where
```

lemma mem_submodule_iff_mem_submoduleToLp[\[source\]](#)

```
lemma mem_submodule_iff_mem_submoduleToLp (f : E × E) :
  f ∈ M ↔ (WithLp.toLp 2 f) ∈ submoduleToLp M
```

lemma submoduleToLp_closure[\[source\]](#)

```
lemma submoduleToLp_closure :
  (submoduleToLp M.topologicalClosure) = (submoduleToLp M).topologicalClosure
```

lemma mem_submodule_closure_iff_mem_submoduleToLp_closure[\[source\]](#)

```
lemma mem_submodule_closure_iff_mem_submoduleToLp_closure (f : E × E) :
  f ∈ M.topologicalClosure ↔ (WithLp.toLp 2 f) ∈ (submoduleToLp M).topologicalClosure
```

lemma mem_submodule_adjoint_iff_mem_submoduleToLp_orthogonal[\[source\]](#)

```
lemma mem_submodule_adjoint_iff_mem_submoduleToLp_orthogonal (f : E × E) :
  f ∈ M.adjoint ↔ WithLp.toLp 2 (f.2, -f.1) ∈ (submoduleToLp M)⊥
```

lemma mem_submodule_adjoint_adjoint_iff_mem_submoduleToLp_orthogonal_orthogonal[\[source\]](#)

```
lemma mem_submodule_adjoint_adjoint_iff_mem_submoduleToLp_orthogonal_orthogonal (f : E × E) :
  f ∈ M.adjoint.adjoint ↔ WithLp.toLp 2 f ∈ (submoduleToLp M)⊥⊥
```

lemma mem_submodule_closure_adjoint_iff_mem_submoduleToLp_closure_orthogonal[\[source\]](#)

```
lemma mem_submodule_closure_adjoint_iff_mem_submoduleToLp_closure_orthogonal (f : E × E) :
  f ∈ M.topologicalClosure.adjoint ↔
  WithLp.toLp 2 (f.2, -f.1) ∈ (submoduleToLp M).topologicalClosure⊥
```

2.7 PhysLean.Mathematics.KroneckerDelta

[PhysLean/Mathematics/KroneckerDelta.lean](#) on GitHub.

def kroneckerDelta

[\[source\]](#)

The Kronecker delta function, ite (i = j) 1 0.

```
def kroneckerDelta [Ring R] (i j : Fin d) : R
```

lemma kroneckerDelta_symm

[\[source\]](#)

```
lemma kroneckerDelta_symm [Ring R] (i j : Fin d) :  
  kroneckerDelta (R := R) i j = kroneckerDelta j i
```

lemma kroneckerDelta_self

[\[source\]](#)

```
lemma kroneckerDelta_self [Ring R] : ∀ i : Fin d, kroneckerDelta (R := R) i i = 1
```

2.8 PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.PlusU1.QuadSol

[PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/QuadSol.lean](#) on GitHub.

lemma genericToQuad_ne_zero

[\[source\]](#)

```
lemma genericToQuad_ne_zero (S : (PlusU1 n).QuadSols) (h : α₁ C S.1 ≠ 0) :  
  (α₁ C S.1)⁻¹ • genericToQuad C S.1 = S
```

2.9 PhysLean.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.PlusU1.QuadSolToSol

[PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/QuadSolToSol.lean](#) on GitHub.

lemma quadSolToSolInv_α₁_α₂_ne_zero

[\[source\]](#)

```
lemma quadSolToSolInv_α₁_α₂_ne_zero (S : (PlusU1 n).Sols) (h : α₁ S.1 ≠ 0) :  
  ¬ (α₁ (quadSolToSolInv S).1 = 0 ∧ α₂ (quadSolToSolInv S).1 = 0)
```

2.10 PhysLean.Particles.BeyondTheStandardModel.TwoHDM.GramMatrix

[PhysLean/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean](#) on GitHub.

lemma gramVector_eq

[\[source\]](#)

The lemma manifesting the definitional equality for the gramVector.

```
lemma gramVector_eq (H : TwoHiggsDoublet) : H.gramVector = fun μ =>  
  2 * PauliMatrix.pauliBasis.repr (gramMatrix H, gramMatrix_selfAdjoint H) μ
```

lemma mem_orbit_gaugeGroupI_iff_gramVector

[\[source\]](#)

```
lemma mem_orbit_gaugeGroupI_iff_gramVector (H1 H2 : TwoHiggsDoublet) :  
  H1 ∈ MulAction.orbit GaugeGroupI H2 ↔ H1.gramVector = H2.gramVector
```

2.11 PhysLean.Particles.BeyondTheStandardModel.TwoHDM.Potential

[PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean](#) on GitHub.

instance Zero PotentialParameters

[\[source\]](#)

```
instance : Zero PotentialParameters where
```

lemma ξ_zero

[\[source\]](#)

```
@[simp]
lemma ξ_zero : (0 : PotentialParameters).ξ = 0
```

lemma η_zero

[\[source\]](#)

```
@[simp]
lemma η_zero : (0 : PotentialParameters).η = 0
```

def stabilityCounterExample

[\[source\]](#)

*An example of potential parameters that serve as a counterexample to the stability condition given in [arXiv:hep-ph/0605184](#). This corresponds to the potential: $2 * (\langle H.\phi_1, H.\phi_2 \rangle_{\mathbb{C}}).im + \|H.\phi_1 - H.\phi_2\|^4$ which has the property that the quartic term is non-negative and only zero if the mass term is also zero, but the potential is not stable. In the proof that `stabilityCounterExample_not_potentialIsStable`, we give explicit vectors $H.\phi_1$ and $H.\phi_2$ that show this potential is not stable.*

This is the first occurrence of such a counterexample in the literature to the best of the author's knowledge.

```
def stabilityCounterExample : PotentialParameters
```

lemma stabilityCounterExample_ξ

[\[source\]](#)

```
lemma stabilityCounterExample_ξ :
  stabilityCounterExample.ξ = fun
  | .inl 0 => 0
  | .inr 0 => 0
  | .inr 1 => 1
  | .inr 2 => 0
```

lemma stabilityCounterExample_η

[\[source\]](#)

```
lemma stabilityCounterExample_η :
  stabilityCounterExample.η = fun μ => fun v =>
  match μ, v with
  | .inl 0, .inl 0 => 1
  | .inl 0, .inr 0 => -1
  | .inl 0, .inr 1 => 0
  | .inl 0, .inr 2 => 0
  | .inr 0, .inl 0 => -1
  | .inr 1, .inl 0 => 0
  | .inr 2, .inl 0 => 0
  | .inr 0, .inr 0 => 1
```

```

| .inr 1, .inr 1 => 0
| .inr 2, .inr 2 => 0
| .inr 0, .inr 1 => 0
| .inr 2, .inr 0 => 0
| .inr 2, .inr 1 => 0
| .inr 1, .inr 0 => 0
| .inr 0, .inr 2 => 0
| .inr 1, .inr 2 => 0

```

lemma massTerm_zero[\[source\]](#)

```
@[simp]
```

```
lemma massTerm_zero : massTerm 0 = 0
```

lemma massTerm_stabilityCounterExample[\[source\]](#)

```
lemma massTerm_stabilityCounterExample (H : TwoHiggsDoublet) :
  massTerm PotentialParameters.stabilityCounterExample H =
  2 * ((H.ϕ1, H.ϕ2)₋ℂ).im
```

lemma quarticTerm_zero[\[source\]](#)

```
@[simp]
```

```
lemma quarticTerm_zero : quarticTerm 0 = 0
```

lemma quarticTerm_stabilityCounterExample[\[source\]](#)

```
lemma quarticTerm_stabilityCounterExample (H : TwoHiggsDoublet) :
  quarticTerm .stabilityCounterExample H =
  (||H.ϕ1|| ^ 2 + ||H.ϕ2|| ^ 2 - 2 * ((H.ϕ1, H.ϕ2)₋ℂ).re) ^ 2
```

lemma quarticTerm_stabilityCounterExample_eq_norm_pow_four[\[source\]](#)

```
lemma quarticTerm_stabilityCounterExample_eq_norm_pow_four (H : TwoHiggsDoublet) :
  quarticTerm .stabilityCounterExample H = ||H.ϕ1 - H.ϕ2|| ^ 4
```

lemma quarticTerm_stabilityCounterExample_nonneg[\[source\]](#)

```
lemma quarticTerm_stabilityCounterExample_nonneg (H : TwoHiggsDoublet) :
  0 ≤ quarticTerm .stabilityCounterExample H
```

lemma massTerm_zero_of_quarticTerm_zero_stabilityCounterExample[\[source\]](#)

```
lemma massTerm_zero_of_quarticTerm_zero_stabilityCounterExample (H : TwoHiggsDoublet)
  (h : quarticTerm .stabilityCounterExample H = 0) :
  massTerm .stabilityCounterExample H = 0
```

lemma potential_zero[\[source\]](#)

```
@[simp]
```

```
lemma potential_zero : potential 0 = 0
```

lemma potential_stabilityCounterExample[\[source\]](#)

```
lemma potential_stabilityCounterExample (H : TwoHiggsDoublet) :
  potential .stabilityCounterExample H = 2 * ((H.Φ1, H.Φ2)_C).im + ||H.Φ1 - H.Φ2|| ^ 4
```

lemma potential_eq_gramVector[\[source\]](#)

```
lemma potential_eq_gramVector (P : PotentialParameters) (H : TwoHiggsDoublet) :
  potential P H = ∑ μ, P.ξ μ * H.gramVector μ +
  ∑ a, ∑ b, H.gramVector a * H.gramVector b * P.η a b
```

def PotentialIsStable[\[source\]](#)

The condition that the potential is stable.

```
def PotentialIsStable (P : PotentialParameters) : Prop
```

lemma stabilityCounterExample_not_potentialIsStable[\[source\]](#)

The potential stabilityCounterExample is not stable.

```
lemma stabilityCounterExample_not_potentialIsStable :
  ¬ PotentialIsStable .stabilityCounterExample
```

lemma massTermReduced_zero[\[source\]](#)

```
@[simp]
lemma massTermReduced_zero : massTermReduced 0 = 0
```

lemma massTermReduced_stabilityCounterExample[\[source\]](#)

```
lemma massTermReduced_stabilityCounterExample (k : EuclideanSpace ℝ (Fin 3)) :
  massTermReduced .stabilityCounterExample k = k 1
```

lemma quarticTermReduced_zero[\[source\]](#)

```
@[simp]
lemma quarticTermReduced_zero : quarticTermReduced 0 = 0
```

lemma quarticTermReduced_stabilityCounterExample[\[source\]](#)

```
lemma quarticTermReduced_stabilityCounterExample (k : EuclideanSpace ℝ (Fin 3)) :
  quarticTermReduced .stabilityCounterExample k = (1 - k 0) ^ 2
```

lemma quarticTermReduced_stabilityCounterExample_nonneg[\[source\]](#)

```
lemma quarticTermReduced_stabilityCounterExample_nonneg (k : EuclideanSpace ℝ (Fin 3)) :
  0 ≤ quarticTermReduced .stabilityCounterExample k
```

lemma potentialIsStable_iff_forall_gramVector[\[source\]](#)

```

lemma potentialIsStable_iff_forall_gramVector (P : PotentialParameters) :
  PotentialIsStable P ↔ ∃ c : ℝ, ∀ K : Fin 1 ⊕ Fin 3 → ℝ, 0 ≤ K (Sum.inl 0) →
    ∑ μ : Fin 3, K (Sum.inr μ) ^ 2 ≤ K (Sum.inl 0) ^ 2 →
    c ≤ ∑ μ, P.ξ μ * K μ + ∑ a, ∑ b, K a * K b * P.η a b

```

lemma potentialIsStable_iff_forall_euclid[\[source\]](#)

```

lemma potentialIsStable_iff_forall_euclid (P : PotentialParameters) :
  PotentialIsStable P ↔ ∃ c, ∀ K0 : ℝ, ∀ K : EuclideanSpace ℝ (Fin 3), 0 ≤ K0 →
    ||K|| ^ 2 ≤ K0 ^ 2 → c ≤ P.ξ (Sum.inl 0) * K0 + ∑ μ, P.ξ (Sum.inr μ) * K μ
    + K0 ^ 2 * P.η (Sum.inl 0) (Sum.inl 0)
    + 2 * K0 * ∑ b, K b * P.η (Sum.inl 0) (Sum.inr b) +
    ∑ a, ∑ b, K a * K b * P.η (Sum.inr a) (Sum.inr b)

```

lemma potentialIsStable_iff_forall_euclid_lt[\[source\]](#)

```

lemma potentialIsStable_iff_forall_euclid_lt (P : PotentialParameters) :
  PotentialIsStable P ↔ ∃ c ≤ 0, ∀ K0 : ℝ, ∀ K : EuclideanSpace ℝ (Fin 3), 0 < K0 →
    ||K|| ^ 2 ≤ K0 ^ 2 → c ≤ P.ξ (Sum.inl 0) * K0 + ∑ μ, P.ξ (Sum.inr μ) * K μ
    + K0 ^ 2 * P.η (Sum.inl 0) (Sum.inl 0)
    + 2 * K0 * ∑ b, K b * P.η (Sum.inl 0) (Sum.inr b) +
    ∑ a, ∑ b, K a * K b * P.η (Sum.inr a) (Sum.inr b)

```

lemma potentialIsStable_iff_exists_forall_forall_reduced[\[source\]](#)

```

lemma potentialIsStable_iff_exists_forall_forall_reduced (P : PotentialParameters) :
  PotentialIsStable P ↔ ∃ c ≤ 0, ∀ K0 : ℝ, ∀ k : EuclideanSpace ℝ (Fin 3), 0 < K0 →
    ||k|| ^ 2 ≤ 1 → c ≤ K0 * massTermReduced P k + K0 ^ 2 * quarticTermReduced P k

```

lemma quarticTermReduced_nonneg_of_potentialIsStable[\[source\]](#)

```

lemma quarticTermReduced_nonneg_of_potentialIsStable (P : PotentialParameters)
  (hP : PotentialIsStable P) (k : EuclideanSpace ℝ (Fin 3))
  (hk : ||k|| ^ 2 ≤ 1) : 0 ≤ quarticTermReduced P k

```

lemma potentialIsStable_iff_massTermReduced_sq_le_quarticTermReduced[\[source\]](#)

```

lemma potentialIsStable_iff_massTermReduced_sq_le_quarticTermReduced (P : PotentialParameters)
  :
  PotentialIsStable P ↔ ∃ c, 0 ≤ c ∧ ∀ k : EuclideanSpace ℝ (Fin 3), ||k|| ^ 2 ≤ 1 →
    0 ≤ quarticTermReduced P k ∧
    (massTermReduced P k < 0 →
      massTermReduced P k ^ 2 ≤ 4 * quarticTermReduced P k * c)

```

lemma massTermReduced_pos_of_quarticTermReduced_zero_potentialIsStable[\[source\]](#)

```

lemma massTermReduced_pos_of_quarticTermReduced_zero_potentialIsStable (P : PotentialParameters)
  )
  (hP : PotentialIsStable P) (k : EuclideanSpace ℝ (Fin 3))
  (hk : ||k|| ^ 2 ≤ 1) (hq : quarticTermReduced P k = 0) : 0 ≤ massTermReduced P k

```

lemma potentialIsStable_of_strong[\[source\]](#)

The potential is stable if it is strongly stable, i.e. its quartic term is always positive. The proof of this result relies on the compactness of the closed unit ball in EuclideanSpace $\mathbb{R}$ (Fin 3), and the extreme value theorem.

```
lemma potentialIsStable_of_strong (P : PotentialParameters)
  (h :  $\forall k, \|k\|^2 \leq 1 \rightarrow 0 < \text{quarticTermReduced } P \ k$ ) :
  PotentialIsStable P
```

lemma forall_reduced_exists_not_potentialIsStable[\[source\]](#)

A lemma invalidating the step in <https://arxiv.org/pdf/hep-ph/0605184> leading to equation (4.4).

```
lemma forall_reduced_exists_not_potentialIsStable :
   $\exists P, \neg \text{PotentialIsStable } P \wedge (\forall k : \text{EuclideanSpace } \mathbb{R} \text{ (Fin 3)}, \|k\|^2 \leq 1 \rightarrow$ 
   $0 \leq \text{quarticTermReduced } P \ k \wedge (\text{quarticTermReduced } P \ k = 0 \rightarrow 0 \leq \text{massTermReduced } P \ k))$ 
```

2.12 PhysLean.Particles.FlavorPhysics.CKMMatrix.Relations

[PhysLean/Particles/FlavorPhysics/CKMMatrix/Relations.lean](#) on GitHub.

lemma ud_us_ne_zero_iff_ub_ne_one[\[source\]](#)

```
lemma ud_us_ne_zero_iff_ub_ne_one (V : CKMMatrix) :
   $[V]_{ud} \neq 0 \vee [V]_{us} \neq 0 \leftrightarrow \text{norm } [V]_{ub} \neq 1$ 
```

lemma normSq_Vud_plus_normSq_Vus_ne_zero_ℝ[\[source\]](#)

```
lemma normSq_Vud_plus_normSq_Vus_ne_zero_ℝ {V : CKMMatrix} (hb :  $[V]_{ud} \neq 0 \vee [V]_{us} \neq 0$ ) :
   $\text{normSq } [V]_{ud} + \text{normSq } [V]_{us} \neq 0$ 
```

lemma VAbsub_ne_zero_Vud_Vus_ne_zero[\[source\]](#)

```
lemma VAbsub_ne_zero_Vud_Vus_ne_zero {V : Quotient CKMMatrixSetoid}
  (hV :  $V_{\text{Abs}} 0 \ 2 \ V \neq 1$ ) :  $(V_{ud} \text{Abs } V^2 + V_{us} \text{Abs } V^2) \neq 0$ 
```

lemma VAbsub_ne_zero_sqrt_Vud_Vus_ne_zero[\[source\]](#)

```
lemma VAbsub_ne_zero_sqrt_Vud_Vus_ne_zero {V : Quotient CKMMatrixSetoid}
  (hV :  $V_{\text{Abs}} 0 \ 2 \ V \neq 1$ ) :  $\sqrt{(V_{ud} \text{Abs } V^2 + V_{us} \text{Abs } V^2)} \neq 0$ 
```

lemma normSq_Vud_plus_normSq_Vus_ne_zero_ℂ[\[source\]](#)

```
lemma normSq_Vud_plus_normSq_Vus_ne_zero_ℂ {V : CKMMatrix} (hb :  $[V]_{ud} \neq 0 \vee [V]_{us} \neq 0$ ) :
   $(\text{normSq } [V]_{ud} : \mathbb{C}) + \text{normSq } [V]_{us} \neq 0$ 
```

lemma Vabs_sq_add_ne_zero[\[source\]](#)

```
lemma Vabs_sq_add_ne_zero {V : CKMMatrix} (hb :  $[V]_{ud} \neq 0 \vee [V]_{us} \neq 0$ ) :
   $((V_{ud} \text{Abs } [V]) : \mathbb{C}) * \uparrow(V_{ud} \text{Abs } [V]) + \uparrow(V_{us} \text{Abs } [V]) * \uparrow(V_{us} \text{Abs } [V]) \neq 0$ 
```


lemma cb_tb_ne_zero_iff_ub_ne_one[\[source\]](#)

```
lemma cb_tb_ne_zero_iff_ub_ne_one (V : CKMMatrix) :
  [V]cb ≠ 0 ∧ [V]tb ≠ 0 ↔ norm [V]ub ≠ 1
```

2.13 PhysLean.Particles.FlavorPhysics.CKMMatrix.StandardParameterization.StandardParameters

[PhysLean/Particles/FlavorPhysics/CKMMatrix/StandardParameterization/StandardParameters.lean](#) on GitHub.

lemma S₂₃_of_Vub_ne_one[\[source\]](#)

```
lemma S23_of_Vub_ne_one {V : Quotient CKMMatrixSetoid} (ha : VubAbs V ≠ 1) :
  S23 V = VcbAbs V / √ (VudAbs V ^ 2 + VusAbs V ^ 2)
```

lemma C₂₃_of_Vub_ne_one[\[source\]](#)

```
lemma C23_of_Vub_ne_one {V : Quotient CKMMatrixSetoid} (ha : VubAbs V ≠ 1) :
  C23 V = VtbAbs V / √ (VudAbs V ^ 2 + VusAbs V ^ 2)
```

lemma mulExpδ₁₃_on_param_ne_zero_arg[\[source\]](#)

```
lemma mulExpδ13_on_param_ne_zero_arg (V : CKMMatrix) (δ13 : ℝ)
  (h1 : mulExpδ13 [[standParam (θ12 [V]) (θ13 [V]) (θ23 [V]) δ13]] ≠ 0) :
  cexp (arg (mulExpδ13 [[standParam (θ12 [V]) (θ13 [V]) (θ23 [V]) δ13]]) * I) =
  cexp (δ13 * I)
```

2.14 PhysLean.Particles.StandardModel.AnomalyCancellation.NoGrav.One.Lemmas

[PhysLean/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Lemmas.lean](#) on GitHub.

lemma accGrav_Q_ne_zero[\[source\]](#)

For a set of 1-family SM charges satisfying all ACCs except the gravitational, if the Q charge is not zero then the charges satisfy the gravitational ACCs.

```
lemma accGrav_Q_ne_zero {S : (SMNoGrav 1).Sols} (hQ : Q S.val (0 : Fin 1) ≠ 0) :
  accGrav S.val = 0
```

2.15 PhysLean.Particles.SuperSymmetry.SU5.ChargeSpectrum.MinimalSuperSet

[PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimalSuperSet.lean](#) on GitHub.

lemma self_ne_mem_minimalSuperSet[\[source\]](#)

```
lemma self_ne_mem_minimalSuperSet (S5 S10 : Finset ℤ) (x y : ChargeSpectrum ℤ)
  (hy : y ∈ minimalSuperSet S5 S10 x) : x ≠ y
```

2.16 PhysLean.QFT.PerturbationTheory.CreateAnnihilate

[PhysLean/QFT/PerturbationTheory/CreateAnnihilate.lean](#) on GitHub.

instance Std.Total normalOrder

[\[source\]](#)

Normal ordering is total.

instance : Std.Total normalOrder **where**

2.17 PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommute

[PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/SuperCommute.lean](#) on GitHub.

lemma statistic_ne_of_superCommuteF_fermionic

[\[source\]](#)

lemma statistic_ne_of_superCommuteF_fermionic { $\phi s \phi s' : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ }
 (h : [ofCrAnListF ϕs , ofCrAnListF $\phi s'$]_sF ∈ statisticSubmodule fermionic) :
 ($\mathcal{F} \mid>_s \phi s$) ≠ ($\mathcal{F} \mid>_s \phi s'$) ∨ [ofCrAnListF ϕs , ofCrAnListF $\phi s'$]_sF = 0

2.18 PhysLean.QFT.PerturbationTheory.FieldSpecification.NormalOrder

[PhysLean/QFT/PerturbationTheory/FieldSpecification/NormalOrder.lean](#) on GitHub.

instance Std.Total $\mathcal{F}.$ normalOrderRel

[\[source\]](#)

Normal ordering is total.

instance : Std.Total $\mathcal{F}.$ normalOrderRel **where**

2.19 PhysLean.QFT.PerturbationTheory.FieldSpecification.TimeOrder

[PhysLean/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean](#) on GitHub.

instance Std.Total $\mathcal{F}.$ timeOrderRel

[\[source\]](#)

Time ordering is total.

instance : Std.Total $\mathcal{F}.$ timeOrderRel **where**

instance Std.Total $\mathcal{F}.$ crAnTimeOrderRel

[\[source\]](#)

Time ordering of CrAnFieldOp is total.

instance : Std.Total $\mathcal{F}.$ crAnTimeOrderRel **where**

instance Std.Total $\mathcal{F}.$ normTimeOrderRel

[\[source\]](#)

Norm-Time ordering of CrAnFieldOp is total.

instance : Std.Total $\mathcal{F}.$ normTimeOrderRel **where**

2.20 PhysLean.QFT.PerturbationTheory.FieldStatistics.Basic

[PhysLean/QFT/PerturbationTheory/FieldStatistics/Basic.lean](#) on GitHub.

lemma bosonic_ne_iff_fermionic_eq[\[source\]](#)

@[simp]

lemma bosonic_ne_iff_fermionic_eq (a : FieldStatistic) : $\neg \text{bosonic} = a \leftrightarrow \text{fermionic} = a$ **lemma fermionic_ne_iff_bosonic_eq**[\[source\]](#)

@[simp]

lemma fermionic_ne_iff_bosonic_eq (a : FieldStatistic) : $\neg \text{fermionic} = a \leftrightarrow \text{bosonic} = a$ **2.21 PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeContraction**[PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeContraction.lean](#) on GitHub.**lemma timeOrder_timeContract_ne_time**[\[source\]](#)

lemma timeOrder_timeContract_ne_time { $\phi \psi : \mathcal{F}.\text{FieldOp}$ }
 (h1 : $\neg (\text{timeOrderRel } \phi \psi \wedge \text{timeOrderRel } \psi \phi)$) :
 $\mathcal{T}(\text{timeContract } \phi \psi) = 0$

2.22 PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeOrder[PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeOrder.lean](#) on GitHub.**lemma ι_timeOrderF_superCommuteF_ne_time**[\[source\]](#)

lemma ι_timeOrderF_superCommuteF_ne_time { $\phi \psi : \mathcal{F}.\text{CrAnFieldOp}$ }
 (h $\phi\psi$: $\neg (\text{crAnTimeOrderRel } \phi \psi \wedge \text{crAnTimeOrderRel } \psi \phi)$) (a b : $\mathcal{F}.\text{FieldOpFreeAlgebra}$) :
 $\iota \mathcal{T}^f(a * [\text{ofCrAnOpF } \phi, \text{ofCrAnOpF } \psi]_s F * b) = 0$

lemma timeOrder_superCommute_ne_time[\[source\]](#)

lemma timeOrder_superCommute_ne_time { $\phi \psi : \mathcal{F}.\text{CrAnFieldOp}$ }
 (h $\phi\psi$: $\neg (\text{crAnTimeOrderRel } \phi \psi \wedge \text{crAnTimeOrderRel } \psi \phi)$) :
 $\mathcal{T}([\text{ofCrAnOp } \phi, \text{ofCrAnOp } \psi]_s) = 0$

lemma timeOrder_superCommute_anPart_ofFieldOp_ne_time[\[source\]](#)

lemma timeOrder_superCommute_anPart_ofFieldOp_ne_time { $\phi \psi : \mathcal{F}.\text{FieldOp}$ }
 (h $\phi\psi$: $\neg (\text{timeOrderRel } \phi \psi \wedge \text{timeOrderRel } \psi \phi)$) :
 $\mathcal{T}([\text{anPart } \phi, \text{ofFieldOp } \psi]_s) = 0$

2.23 PhysLean.QFT.PerturbationTheory.WickContraction.Basic[PhysLean/QFT/PerturbationTheory/WickContraction/Basic.lean](#) on GitHub.**lemma self_ne_getDual?_get**[\[source\]](#)

@[simp]

lemma self_ne_getDual?_get (i : Fin n) (h : (c.getDual? i).isSome) :
 $\neg i = (c.getDual? i).get h$

lemma fstFieldOfContract_ne_sndFieldOfContract[\[source\]](#)

```
lemma fstFieldOfContract_ne_sndFieldOfContract (c : WickContraction n) (a : c.1) :
  c.fstFieldOfContract a ≠ c.sndFieldOfContract a
```

2.24 PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContractNat

[PhysLean/QFT/PerturbationTheory/WickContraction/InsertAndContractNat.lean](#) on GitHub.

lemma insertAndContractNat_some_getDual?_ne_none[\[source\]](#)

```
lemma insertAndContractNat_some_getDual?_ne_none (c : WickContraction n) (i : Fin n.succ)
  (j : c.uncontracted) (k : Fin n) (hkj : k ≠ j.1) :
  (insertAndContractNat c i (some j)).getDual? (i.succAbove k) = none ↔ c.getDual? k = none
```

lemma insertAndContractNat_some_getDual?_ne_isSome[\[source\]](#)

```
lemma insertAndContractNat_some_getDual?_ne_isSome (c : WickContraction n) (i : Fin n.succ)
  (j : c.uncontracted) (k : Fin n) (hkj : k ≠ j.1) :
  ((insertAndContractNat c i (some j)).getDual? (i.succAbove k)).isSome ↔
  (c.getDual? k).isSome
```

lemma insertAndContractNat_some_getDual?_ne_isSome_get[\[source\]](#)

```
lemma insertAndContractNat_some_getDual?_ne_isSome_get (c : WickContraction n) (i : Fin n.succ)
  (j : c.uncontracted) (k : Fin n) (hkj : k ≠ j.1)
  (h : ((insertAndContractNat c i (some j)).getDual? (i.succAbove k)).isSome) :
  ((insertAndContractNat c i (some j)).getDual? (i.succAbove k)).get h =
  i.succAbove ((c.getDual? k).get
    (by simp [hkj, insertAndContractNat_some_getDual?_ne_isSome] using h))
```

2.25 PhysLean.QFT.PerturbationTheory.WickContraction.Join

[PhysLean/QFT/PerturbationTheory/WickContraction/Join.lean](#) on GitHub.

lemma jointLiftLeft_ne_joinLiftRight[\[source\]](#)

```
lemma jointLiftLeft_ne_joinLiftRight {ϕs : List ℱ.FieldOp} {ϕsΛ : WickContraction ϕs.length}
  {ϕsucΛ : WickContraction [ϕsΛ]uc.length} (a : ϕsΛ.1) (b : ϕsucΛ.1) :
  joinLiftLeft a ≠ joinLiftRight b
```

2.26 PhysLean.QFT.PerturbationTheory.WickContraction.Singleton

[PhysLean/QFT/PerturbationTheory/WickContraction/Singleton.lean](#) on GitHub.

lemma singleton_uncontractedEmd_ne_left[\[source\]](#)

```
lemma singleton_uncontractedEmd_ne_left {ϕs : List ℱ.FieldOp} {i j : Fin ϕs.length} (hij : i <
  j)
  (a : Fin [singleton hij]uc.length) :
  (singleton hij).uncontractedListEmd a ≠ i
```

lemma singleton_uncontractedEmd_ne_right[\[source\]](#)

```

lemma singleton_uncontractedEmd_ne_right { $\phi$ s : List  $\mathcal{F}$ .FieldOp} {i j : Fin  $\phi$ s.length} (hij : i < j)
  (a : Fin [singleton hij]uc.length) :
  (singleton hij).uncontractedListEmd a  $\neq$  j

```

2.27 PhysLean.QFT.PerturbationTheory.WickContraction.UncontractedList

[PhysLean/QFT/PerturbationTheory/WickContraction/UncontractedList.lean](#) on GitHub.

lemma uncontractedListOrderPos_le_length[\[source\]](#)

```

@[simp]
lemma uncontractedListOrderPos_le_length (c : WickContraction n) (i : Fin n.succ) :
  c.uncontractedListOrderPos i  $\leq$  c.uncontractedList.length

```

2.28 PhysLean.QuantumMechanics.DDimensions.Hydrogen.Basic

[PhysLean/QuantumMechanics/DDimensions/Hydrogen/Basic.lean](#) on GitHub.

structure HydrogenAtom[\[source\]](#)

A hydrogen atom is characterized by the number of spatial dimensions d , the mass m and the coefficient k for the $1/r$ potential.

```

structure HydrogenAtom where
  d :  $\mathbb{N}$ 
  m :  $\mathbb{R}$ 
  hm :  $0 < m$ 
  k :  $\mathbb{R}$ 

```

lemma m_ne_zero[\[source\]](#)

```

@[simp]
lemma m_ne_zero : H.m  $\neq$  0

```

def hamiltonianReg[\[source\]](#)

The hydrogen atom Hamiltonian regularized by $\varepsilon > 0$ is defined to be $\mathbf{H}(\varepsilon) = (2m)^{-1}\mathbf{p}^2 - k(\mathbf{r}(\varepsilon))^{-1} + \frac{1}{2}\varepsilon^2\mathbf{r}(\varepsilon)^{-3}$.

```

def hamiltonianReg ( $\varepsilon$  :  $\mathbb{R}$ ) :  $\mathcal{S}(\text{Space } H.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } H.d, \mathbb{C})$ 

```

2.29 PhysLean.QuantumMechanics.DDimensions.Hydrogen.LaplaceRungeLenzVector

[PhysLean/QuantumMechanics/DDimensions/Hydrogen/LaplaceRungeLenzVector.lean](#) on GitHub.

def lrlOperator[\[source\]](#)

The (regularized) Laplace-Runge-Lenz vector operator for the d -dimensional hydrogen atom, $\mathbf{A}(\varepsilon)_i = \frac{1}{2}(\mathbf{p}_j \mathbf{L}_{ij} + \mathbf{L}_{ij} \mathbf{p}_j) - mk \cdot \mathbf{r}(\varepsilon)^{-1} \mathbf{x}_i$.

```

def lrlOperator ( $\varepsilon$  :  $\mathbb{R}$ ) (i : Fin H.d) :  $\mathcal{S}(\text{Space } H.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } H.d, \mathbb{C})$ 

```

def lrlOperatorSqr

[\[source\]](#)

The square of the LRL vector operator, $\mathbf{A}(\varepsilon)^2 = \mathbf{A}(\varepsilon)_i \mathbf{A}(\varepsilon)_i$.

```
def lrlOperatorSqr (ε : ℝ) : S(Space H.d, ℂ) → L[ℂ] S(Space H.d, ℂ)
```

lemma lrlOperator_eq

[\[source\]](#)

$$\mathbf{A}(\varepsilon)_i = \mathbf{x}_i \mathbf{p}^2 - (\mathbf{x}_j \mathbf{p}_j) \mathbf{p}_i + \frac{1}{2} i \hbar (d-1) \mathbf{p}_i - m k \cdot \mathbf{r}(\varepsilon)^{-1} \mathbf{x}_i$$

```
lemma lrlOperator_eq (i : Fin H.d) :
  H.lrlOperator ε i = x[i] ∘L p^2 - (∑ j, x[j] ∘L p[j]) ∘L p[i]
  + (2^-1 * Complex.I * ħ * (H.d - 1)) • p[i] - (H.m * H.k) • r[ε, -1] ∘L x[i]
```

lemma lrlOperator_eq'

[\[source\]](#)

$$\mathbf{A}(\varepsilon)_i = \mathbf{L}_{ij} \mathbf{p}_j + \frac{1}{2} i \hbar (d-1) \mathbf{p}_i - m k \cdot \mathbf{r}(\varepsilon)^{-1} \mathbf{x}_i$$

```
lemma lrlOperator_eq' (i : Fin H.d) : H.lrlOperator ε i = ∑ j, L[i, j] ∘L p[j]
  + (2^-1 * Complex.I * ħ * (H.d - 1)) • p[i] - (H.m * H.k) • r[ε, -1] ∘L x[i]
```

lemma lrlOperator_eq''

[\[source\]](#)

$$\mathbf{A}(\varepsilon)_i = \mathbf{p}_j \mathbf{L}_{ij} - \frac{1}{2} i \hbar (d-1) \mathbf{p}_i - m k \cdot \mathbf{r}(\varepsilon)^{-1} \mathbf{x}_i$$

```
lemma lrlOperator_eq'' (i : Fin H.d) : H.lrlOperator ε i = ∑ j, p[j] ∘L L[i, j]
  - (2^-1 * Complex.I * ħ * (H.d - 1)) • p[i] - (H.m * H.k) • r[ε, -1] ∘L x[i]
```

def positionDotMomentum

[\[source\]](#)

The operator $\mathbf{x}_i \mathbf{p}_i$ on Schwartz maps.

```
private def positionDotMomentum {d}
```

def positionCompMomentumSqr

[\[source\]](#)

The operator $\mathbf{x}_i \mathbf{p}^2$ on Schwartz maps.

```
private def positionCompMomentumSqr {d} (i : Fin d)
```

def positionDotMomentumCompMomentum

[\[source\]](#)

The operator $(\mathbf{x}_j \mathbf{p}_j) \mathbf{x}_i$ on Schwartz maps.

```
private def positionDotMomentumCompMomentum {d} (i : Fin d)
```

def constMomentum

[\[source\]](#)

The operator $\frac{1}{2} i \hbar (d-1) \mathbf{p}_i$ on Schwartz maps.

```
private def constMomentum {d} (i : Fin d)
```

def constRadiusRegInvCompPosition[\[source\]](#)

The operator $mk \cdot \mathbf{r}(\varepsilon)^{-1} \mathbf{x}_i$ on Schwartz maps.

```
private def constRadiusRegInvCompPosition (ε : ℝ) (i : Fin H.d)
```

lemma angularMomentum_commutation_lrl[\[source\]](#)

$$\{L_{ij}, A(\varepsilon)_k\} = i\hbar(\delta_{ik}A(\varepsilon)_j - \delta_{jk}A(\varepsilon)_i)$$

```
lemma angularMomentum_commutation_lrl (hε : 0 < ε) (i j k : Fin H.d) :
  {L[i,j], H.lrlOperator ε k} = (Complex.I * ħ * δ[i,k]) • H.lrlOperator ε j
  - (Complex.I * ħ * δ[j,k]) • H.lrlOperator ε i
```

lemma angularMomentum_commutation_lrlSqr[\[source\]](#)

$$\{L_{ij}, A(\varepsilon)^2\} = 0$$

```
lemma angularMomentum_commutation_lrlSqr (hε : 0 < ε) (i j : Fin H.d) :
  {L[i,j], H.lrlOperatorSqr ε} = 0
```

lemma angularMomentumSqr_commutation_lrlSqr[\[source\]](#)

$$\{L^2, A(\varepsilon)^2\} = 0$$

```
lemma angularMomentumSqr_commutation_lrlSqr (hε : 0 < ε) :
  {angularMomentumOperatorSqr (d := H.d), H.lrlOperatorSqr ε} = 0
```

lemma positionDotMomentum_commutation_position[\[source\]](#)

```
private lemma positionDotMomentum_commutation_position {d} (i : Fin d) :
  {positionDotMomentum (d := d), x[i]} = (-Complex.I * ħ) • x[i]
```

lemma positionDotMomentum_commutation_momentum[\[source\]](#)

```
private lemma positionDotMomentum_commutation_momentum {d} (i : Fin d) :
  {positionDotMomentum (d := d), p[i]} = (Complex.I * ħ) • p[i]
```

lemma positionDotMomentum_commutation_momentumSqr[\[source\]](#)

```
private lemma positionDotMomentum_commutation_momentumSqr {d} :
  {positionDotMomentum (d := d), momentumOperatorSqr (d := d)} = (2 * Complex.I * ħ) • p^2
```

lemma positionDotMomentum_commutation_radiusRegPow[\[source\]](#)

```
private lemma positionDotMomentum_commutation_radiusRegPow {d} (hε : 0 < ε) :
  {positionDotMomentum (d := d), radiusRegPowOperator (d := d) ε s} =
  (-s * Complex.I * ħ) • (r[ε,s] - ε ^ 2 • r[ε,s-2])
```

lemma positionCompMomentumSqr_comm[\[source\]](#)

```
private lemma positionCompMomentumSqr_comm {d} (i j : Fin d) :
  {positionCompMomentumSqr (d := d) i, positionCompMomentumSqr j} =
  (-2 * Complex.I * ħ) • p^2 •L L[i,j]
```

lemma positionCompMomentumSqr_comm_positionDotMomentumCompMomentum_add [\[source\]](#)

```
private lemma positionCompMomentumSqr_comm_positionDotMomentumCompMomentum_add {d} (i j : Fin d) :
  {positionCompMomentumSqr (d := d) i, positionDotMomentumCompMomentum j} +
  {positionDotMomentumCompMomentum i, positionCompMomentumSqr j} =
  (-Complex.I * ħ) • p2 ∘L L[i,j]
```

lemma positionDotMomentumCompMomentum_comm [\[source\]](#)

```
private lemma positionDotMomentumCompMomentum_comm {d} (i j : Fin d) :
  {positionDotMomentumCompMomentum i, positionDotMomentumCompMomentum j} = 0
```

lemma positionCompMomentumSqr_comm_constMomentum_add [\[source\]](#)

```
private lemma positionCompMomentumSqr_comm_constMomentum_add {d} (i j : Fin d) :
  {positionCompMomentumSqr i, constMomentum j} +
  {constMomentum i, positionCompMomentumSqr j} = 0
```

lemma positionDotMomentumCompMomentum_comm_constMomentum_add [\[source\]](#)

```
private lemma positionDotMomentumCompMomentum_comm_constMomentum_add {d} (i j : Fin d) :
  {positionDotMomentumCompMomentum (d := d) i, constMomentum j} +
  {constMomentum i, positionDotMomentumCompMomentum j} = 0
```

lemma constMomentum_comm_constMomentum [\[source\]](#)

```
private lemma constMomentum_comm_constMomentum {d} (i j : Fin d) :
  {constMomentum i, constMomentum j} = 0
```

lemma positionCompMomentumSqr_comm_constRadiusRegInvCompPosition_add [\[source\]](#)

```
private lemma positionCompMomentumSqr_comm_constRadiusRegInvCompPosition_add
  (hε : 0 < ε) (i j : Fin H.d) :
  {positionCompMomentumSqr i, constRadiusRegInvCompPosition H ε j} +
  {constRadiusRegInvCompPosition H ε i, positionCompMomentumSqr j} =
  - (2 * H.m * H.k * Complex.I * ħ) • r[ε,-1] ∘L L[i,j]
```

lemma positionDotMomentumCompMomentum_comm_constRadiusRegInvCompPosition_add [\[source\]](#)

```
private lemma positionDotMomentumCompMomentum_comm_constRadiusRegInvCompPosition_add
  (hε : 0 < ε) (i j : Fin H.d) :
  {positionDotMomentumCompMomentum i, constRadiusRegInvCompPosition H ε j} +
  {constRadiusRegInvCompPosition H ε i, positionDotMomentumCompMomentum j} =
  (H.m * H.k * Complex.I * ħ * ε ^ 2) • r[ε,-3] ∘L L[i,j]
```

lemma constMomentum_comm_constRadiusRegInvCompPosition_add [\[source\]](#)

```
private lemma constMomentum_comm_constRadiusRegInvCompPosition_add (hε : 0 < ε) (i j : Fin H.d) :
  {constMomentum i, constRadiusRegInvCompPosition H ε j} +
  {constRadiusRegInvCompPosition H ε i, constMomentum j} = 0
```


lemma constRadiusRegInvCompPosition_comm_constRadiusRegInvCompPosition[\[source\]](#)

```
private lemma constRadiusRegInvCompPosition_comm_constRadiusRegInvCompPosition
  (hε : 0 < ε) (i j : Fin H.d) :
  {constRadiusRegInvCompPosition H ε i, constRadiusRegInvCompPosition H ε j} = 0
```

lemma lrl_commutation_lrl[\[source\]](#)

$$\{A(\varepsilon)_i, A(\varepsilon)_j\} = -i\hbar 2m H(\varepsilon)L_{ij}$$

```
lemma lrl_commutation_lrl (hε : 0 < ε) (i j : Fin H.d) : {H.lrlOperator ε i, H.lrlOperator ε j}
  = (-2 * Complex.I * ħ * H.m) • (H.hamiltonianReg ε) •L L[i,j]
```

lemma pSqr_comm_pL_Lp[\[source\]](#)

```
private lemma pSqr_comm_pL_Lp (i : Fin H.d) :
  {momentumOperatorSqr (d := H.d), ∑ j, (p[j] •L L[i,j] + L[i,j] •L p[j])} = 0
```

lemma rr_comm_rx[\[source\]](#)

```
private lemma rr_comm_rx (hε : 0 < ε) (i : Fin H.d) :
  {radiusRegPowOperator (d := H.d) ε (-1) + (2-1 * ε ^ 2) • r[ε,-3], r[ε,-1] •L x[i]} = 0
```

lemma pSqr_comm_rx[\[source\]](#)

```
private lemma pSqr_comm_rx (hε : 0 < ε) (i : Fin H.d) :
  {momentumOperatorSqr (d := H.d), r[ε,-1] •L x[i]} =
  (-2 * Complex.I * ħ) • r[ε,-1] •L p[i]
  + (ħ ^ 2 * (H.d - 3) : ℝ) • r[ε,-3] •L x[i]
  + (3 * ħ ^ 2 * ε ^ 2) • r[ε,-5] •L x[i]
  + (2 * Complex.I * ħ) • r[ε,-3] •L (∑ j, x[j] •L p[j]) •L x[i]
```

lemma rr_comm_pL_Lp[\[source\]](#)

```
private lemma rr_comm_pL_Lp (hε : 0 < ε) (i : Fin H.d) :
  {radiusRegPowOperator (d := H.d) ε (-1) + (2-1 * ε ^ 2) • r[ε,-3],
  ∑ j, (p[j] •L L[i,j] + L[i,j] •L p[j])} =
  (- Complex.I * ħ) •
  (r[ε,-3] + (3 * 2-1 * ε ^ 2) • r[ε,-5]) •L ∑ j, (x[j] •L L[i,j] + L[i,j] •L x[j])
```

lemma xL_Lx_eq[\[source\]](#)

```
private lemma xL_Lx_eq (hε : 0 < ε) (i : Fin H.d) : ∑ j, (x[j] •L L[i,j] + L[i,j] •L x[j]) =
  (2 : ℝ) • (∑ j, x[j] •L p[j]) •L x[i] - (2 : ℝ) • r[ε,2] •L p[i] + (2 * ε ^ 2) • p[i]
  - (Complex.I * ħ * (H.d - 3)) • x[i]
```

lemma hamiltonianReg_commutation_lrl[\[source\]](#)

$$\{H(\varepsilon), A(\varepsilon)_i\} = i\hbar k \varepsilon^2 (\frac{3}{4}r(\varepsilon)^{-5}(x_j L_{ij} + L_{ij} x_j) + 3i\hbar/2 r(\varepsilon)^{-5}x_i + r(\varepsilon)^{-3}p_i)$$

```
lemma hamiltonianReg_commutation_lrl (hε : 0 < ε) (i : Fin H.d) :
  {H.hamiltonianReg ε, H.lrlOperator ε i} = (Complex.I * ħ * H.k * ε ^ 2) •
  ((3 * 4-1 : ℝ) • r[ε,-5] •L ∑ j, (x[j] •L L[i,j] + L[i,j] •L x[j])
  + (3 * 2-1 * Complex.I * ħ) • r[ε,-5] •L x[i] + r[ε,-3] •L p[i])
```

lemma sum_symmetrize[\[source\]](#)

```
private lemma sum_symmetrize (f : Fin H.d → Fin H.d →  $\mathcal{S}(\text{Space } H.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } H.d, \mathbb{C})$ ) :
   $\sum i, \sum j, f \ i \ j = (2 : \mathbb{C})^{-1} \cdot \sum i, \sum j, (f \ i \ j + f \ j \ i)$ 
```

lemma sum_symmetrize'[\[source\]](#)

```
private lemma sum_symmetrize'
  (f : Fin H.d → Fin H.d → Fin H.d →  $\mathcal{S}(\text{Space } H.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } H.d, \mathbb{C})$ ) :
   $\sum i, \sum j, \sum k, f \ i \ j \ k = (2 : \mathbb{C})^{-1} \cdot \sum i, \sum k, \sum j, (f \ i \ j \ k + f \ k \ j \ i)$ 
```

lemma sum_Lpp_zero[\[source\]](#)

```
private lemma sum_Lpp_zero :  $\sum i : \text{Fin } H.d, \sum j, L[i,j] \circ_L p[j] \circ_L p[i] = 0$ 
```

lemma sum_ppl_zero[\[source\]](#)

```
private lemma sum_ppl_zero :  $\sum i : \text{Fin } H.d, \sum j, p[i] \circ_L p[j] \circ_L L[i,j] = 0$ 
```

lemma sum_LppL[\[source\]](#)

```
private lemma sum_LppL :  $\sum i : \text{Fin } H.d, \sum j, \sum k, L[i,j] \circ_L p[j] \circ_L p[k] \circ_L L[i,k] = p^2 \circ_L L^2$ 
```

lemma sum_Lprx[\[source\]](#)

```
private lemma sum_Lprx (hε : 0 < ε) :
   $\sum i : \text{Fin } H.d, \sum j, L[i,j] \circ_L p[j] \circ_L r[\varepsilon, -1] \circ_L x[i] = r[\varepsilon, -1] \circ_L L^2$ 
```

lemma sum_rxpL[\[source\]](#)

```
private lemma sum_rxpL :
   $\sum i : \text{Fin } H.d, \sum j, r[\varepsilon, -1] \circ_L x[i] \circ_L p[j] \circ_L L[i,j] = r[\varepsilon, -1] \circ_L L^2$ 
```

lemma sum_prx[\[source\]](#)

```
private lemma sum_prx (hε : 0 < ε) :  $\sum i : \text{Fin } H.d, p[i] \circ_L r[\varepsilon, -1] \circ_L x[i] =$ 
   $r[\varepsilon, -1] \circ_L \sum i : \text{Fin } H.d, x[i] \circ_L p[i] - (\text{Complex.I} * \hbar * (H.d - 1)) \cdot r[\varepsilon, -1]$ 
   $- (\text{Complex.I} * \hbar * \varepsilon^2) \cdot r[\varepsilon, -3]$ 
```

lemma sum_rxp[\[source\]](#)

```
private lemma sum_rxp :  $\sum i : \text{Fin } H.d, r[\varepsilon, -1] \circ_L x[i] \circ_L p[i] =$ 
   $r[\varepsilon, -1] \circ_L \sum i : \text{Fin } H.d, x[i] \circ_L p[i]$ 
```

lemma sum_rrx[\[source\]](#)

```
private lemma sum_rrx (hε : 0 < ε) :  $\sum i, r[\varepsilon, -1] \circ_L x[i] \circ_L r[\varepsilon, -1] \circ_L x[i] =$ 
   $\text{ContinuousLinearMap.id } \mathbb{C} \mathcal{S}(\text{Space } H.d, \mathbb{C}) - (\varepsilon^2) \cdot r[\varepsilon, -2]$ 
```

lemma lrlOperatorSqr_eq[\[source\]](#)

The square of the (regularized) LRL vector operator is related to the (regularized) Hamiltonian $\mathbf{H}(\varepsilon)$ of the hydrogen atom, square of the angular momentum $\mathbf{L}^2$ and powers of $\mathbf{r}(\varepsilon)$ as $\mathbf{A}(\varepsilon)^2 = 2m \mathbf{H}(\varepsilon)(\mathbf{L}^2 + \frac{1}{4}\hbar^2(d-1)^2) + m^2 k^2 - m^2 k^2 \varepsilon^2 \mathbf{r}(\varepsilon)^{-2} + m k \varepsilon^2 \mathbf{r}(\varepsilon)^{-3}(\mathbf{L}^2 + \frac{1}{4}\hbar^2(d-1)(d-3))$.

```
lemma lrlOperatorSqr_eq (hε : 0 < ε) : H.lrlOperatorSqr ε =
  (2 * H.m) • (H.hamiltonianReg ε) • L
  (L^2 + (4^-1 * ħ ^ 2 * (H.d - 1) ^ 2 : ℝ) • ContinuousLinearMap.id ℂ S(Space H.d, ℂ))
+ (H.m * H.k) ^ 2 • ContinuousLinearMap.id ℂ S(Space H.d, ℂ)
- ((H.m * H.k) ^ 2 * ε ^ 2) • r[ε,-2]
+ (H.m * H.k * ε ^ 2) • r[ε,-3] • L
  (L^2 + (4^-1 * ħ^2 * (H.d - 1) * (H.d - 3) : ℝ) •
  ContinuousLinearMap.id ℂ S(Space H.d, ℂ))
```

2.30 PhysLean.QuantumMechanics.DDimensions.Operators.AngularMomentum

[PhysLean/QuantumMechanics/DDimensions/Operators/AngularMomentum.lean](#) on GitHub.

def angularMomentumOperator[\[source\]](#)

Component $i\ j$ of the angular momentum operator is the continuous linear map from $\mathcal{S}(\text{Space } d, \mathbb{C})$ to itself defined by $\mathbf{L}_{ij} = \mathbf{x}_i \circ \mathbf{p}_j - \mathbf{x}_j \circ \mathbf{p}_i$.

```
def angularMomentumOperator {d : ℕ} (i j : Fin d) : S(Space d, ℂ) → L[ℂ] S(Space d, ℂ)
```

lemma angularMomentumOperator_apply_fun[\[source\]](#)

```
lemma angularMomentumOperator_apply_fun {d : ℕ} (i j : Fin d) (ψ : S(Space d, ℂ)) :
  L[i,j] ψ = x[i] (p[j] ψ) - x[j] (p[i] ψ)
```

lemma angularMomentumOperator_apply[\[source\]](#)

```
lemma angularMomentumOperator_apply {d : ℕ} (i j : Fin d) (ψ : S(Space d, ℂ)) (x : Space d) :
  L[i,j] ψ x = x[i] (p[j] ψ) x - x[j] (p[i] ψ) x
```

def angularMomentumOperatorSqr[\[source\]](#)

The square of the angular momentum operator, $\mathbf{L}^2 = \frac{1}{2} \sum_{i,j} \mathbf{L}_{ij} \circ \mathbf{L}_{ij}$.

```
def angularMomentumOperatorSqr {d : ℕ} : S(Space d, ℂ) → L[ℂ] S(Space d, ℂ)
```

lemma angularMomentumOperatorSqr_apply_fun[\[source\]](#)

```
lemma angularMomentumOperatorSqr_apply_fun {d : ℕ} (ψ : S(Space d, ℂ)) :
  L^2 ψ = (2 : ℂ)^-1 • ∑ i, ∑ j, L[i,j] (L[i,j] ψ)
```

lemma angularMomentumOperatorSqr_apply[\[source\]](#)

```
lemma angularMomentumOperatorSqr_apply {d : ℕ} (ψ : S(Space d, ℂ)) (x : Space d) :
  L^2 ψ x = (2 : ℂ)^-1 * ∑ i, ∑ j, L[i,j] (L[i,j] ψ) x
```

lemma angularMomentumOperator_antisymm[\[source\]](#)

The angular momentum operator is antisymmetric, $\mathbf{L}_{ij} = -\mathbf{L}_{ji}$

```
lemma angularMomentumOperator_antisymm {d : ℕ} (i j : Fin d) : L[i,j] = - L[j,i]
```

lemma angularMomentumOperator_eq_zero[\[source\]](#)

Angular momentum operator components with repeated index vanish, $\mathbf{L}_{ii} = 0$.

```
lemma angularMomentumOperator_eq_zero {d : ℕ} (i : Fin d) : L[i,i] = 0
```

lemma angularMomentumOperator1D_trivial[\[source\]](#)

In one dimension the angular momentum operator is trivial.

```
lemma angularMomentumOperator1D_trivial : ∀ (i j : Fin 1), L[i,j] = 0
```

def angularMomentumOperator2D[\[source\]](#)

The angular momentum (pseudo)scalar operator in two dimensions, $\mathbf{L} = \mathbf{L}_{\theta 1}$.

```
def angularMomentumOperator2D : S(Space 2, ℂ) → L[ℂ] S(Space 2, ℂ)
```

def angularMomentumOperator3D[\[source\]](#)

The angular momentum (pseudo)vector operator in three dimension, $\mathbf{L}_i = \frac{1}{2} \sum_{j,k} \epsilon_{ijk} \mathbf{L}_{jk}$.

```
def angularMomentumOperator3D (i : Fin 3) : S(Space 3, ℂ) → L[ℂ] S(Space 3, ℂ)
```

2.31 PhysLean.QuantumMechanics.DDimensions.Operators.Commutation

[PhysLean/QuantumMechanics/DDimensions/Operators/Commutation.lean](#) on GitHub.

lemma ite_cond_symm[\[source\]](#)

```
private lemma ite_cond_symm (i j : Fin d) :  
  (if i = j then A else B) = (if j = i then A else B)
```

lemma leibniz_lie[\[source\]](#)

```
lemma leibniz_lie {d : ℕ} (A B C : S(Space d, ℂ) → L[ℂ] S(Space d, ℂ)) :  
  {A ∘L B, C} = A ∘L {B, C} + {A, C} ∘L B
```

lemma lie_leibniz[\[source\]](#)

```
lemma lie_leibniz {d : ℕ} (A B C : S(Space d, ℂ) → L[ℂ] S(Space d, ℂ)) :  
  {A, B ∘L C} = B ∘L {A, C} + {A, B} ∘L C
```

lemma comp_eq_comp_add_commute[\[source\]](#)

lemma comp_eq_comp_add_commute (A B : $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow \mathcal{L}[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$) :
 $A \circ_L B = B \circ_L A + [A, B]$

lemma comp_eq_comp_sub_commute[\[source\]](#)

lemma comp_eq_comp_sub_commute (A B : $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow \mathcal{L}[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$) :
 $A \circ_L B = B \circ_L A - [B, A]$

lemma position_commutation_position[\[source\]](#)

Position operators commute: $[x_i, x_j] = 0$.

lemma position_commutation_position {d : $\mathbb{N}$ } (i j : Fin d) : {x[i], x[j]} = 0

lemma position_comp_commute[\[source\]](#)

lemma position_comp_commute {d : $\mathbb{N}$ } (i j : Fin d) : x[i] $\circ_L$ x[j] = x[j] $\circ_L$ x[i]

lemma position_commutation_radiusRegPow[\[source\]](#)

lemma position_commutation_radiusRegPow (hε : 0 < ε) (i : Fin d) :
 {x[i], radiusRegPowOperator (d := d) ε s} = 0

lemma position_comp_radiusRegPow_commute[\[source\]](#)

lemma position_comp_radiusRegPow_commute (hε : 0 < ε) (i : Fin d) :
 x[i] $\circ_L$ r[ε,s] = r[ε,s] $\circ_L$ x[i]

lemma radiusRegPow_commutation_radiusRegPow[\[source\]](#)

lemma radiusRegPow_commutation_radiusRegPow (hε : 0 < ε) :
 {radiusRegPowOperator (d := d) ε s, radiusRegPowOperator (d := d) ε t} = 0

lemma momentum_commutation_momentum[\[source\]](#)

Momentum operators commute: $[p_i, p_j] = 0$.

lemma momentum_commutation_momentum {d : $\mathbb{N}$ } (i j : Fin d) : {p[i], p[j]} = 0

lemma momentum_comp_commute[\[source\]](#)

lemma momentum_comp_commute {d : $\mathbb{N}$ } (i j : Fin d) : p[i] $\circ_L$ p[j] = p[j] $\circ_L$ p[i]

lemma momentumSqr_commutation_momentum[\[source\]](#)

lemma momentumSqr_commutation_momentum {d : $\mathbb{N}$ } (i : Fin d) :
 {momentumOperatorSqr (d := d), p[i]} = 0

lemma momentumSqr_comp_momentum_commute[\[source\]](#)

```
lemma momentumSqr_comp_momentum_commute {d : ℕ} (i : Fin d) : p2 ∘L p[i] = p[i] ∘L p2
```

lemma position_commutation_momentum[\[source\]](#)

The canonical commutation relations: $[x_i, p_j] = i\hbar \delta_{ij}$.

```
lemma position_commutation_momentum {d : ℕ} (i j : Fin d) : {x[i], p[j]} =  
(Complex.I * ħ * δ[i,j]) • ContinuousLinearMap.id ℂ S(Space d, ℂ)
```

lemma momentum_comp_position_eq[\[source\]](#)

```
lemma momentum_comp_position_eq (i j : Fin d) : p[j] ∘L x[i] =  
x[i] ∘L p[j] - (Complex.I * ħ * δ[i,j]) • ContinuousLinearMap.id ℂ S(Space d, ℂ)
```

lemma position_position_commutation_momentum[\[source\]](#)

```
lemma position_position_commutation_momentum {d : ℕ} (i j k : Fin d) : {x[i] ∘L x[j], p[k]} =  
(Complex.I * ħ * δ[i,k]) • x[j] + (Complex.I * ħ * δ[j,k]) • x[i]
```

lemma position_commutation_momentum_momentum[\[source\]](#)

```
lemma position_commutation_momentum_momentum {d : ℕ} (i j k : Fin d) : {x[i], p[j] ∘L p[k]} =  
(Complex.I * ħ * δ[i,k]) • p[j] + (Complex.I * ħ * δ[i,j]) • p[k]
```

lemma position_commutation_momentumSqr[\[source\]](#)

```
lemma position_commutation_momentumSqr {d : ℕ} (i : Fin d) : {x[i], p2} =  
(2 * Complex.I * ħ) • p[i]
```

lemma radiusRegPow_commutation_momentum[\[source\]](#)

```
lemma radiusRegPow_commutation_momentum (hε : 0 < ε) (i : Fin d) :  
{radiusRegPowOperator (d := d) ε s, p[i]} = (s * Complex.I * ħ) • r[ε,s-2] ∘L x[i]
```

lemma momentum_comp_radiusRegPow_eq[\[source\]](#)

```
lemma momentum_comp_radiusRegPow_eq (hε : 0 < ε) (i : Fin d) :  
p[i] ∘L r[ε,s] = r[ε,s] ∘L p[i] - (s * Complex.I * ħ) • r[ε,s-2] ∘L x[i]
```

lemma radiusRegPow_commutation_momentumSqr[\[source\]](#)

```
lemma radiusRegPow_commutation_momentumSqr (hε : 0 < ε) :  
{radiusRegPowOperator (d := d) ε s, momentumOperatorSqr (d := d)} =  
(2 * s * Complex.I * ħ) • r[ε,s-2] ∘L ∑ i, x[i] ∘L p[i]  
+ (s * ħ ^ 2) • ((d + s - 2) • r[ε,s-2] - (ε ^ 2 * (s - 2)) • r[ε,s-4])
```

lemma angularMomentum_commutation_position[\[source\]](#)

```
lemma angularMomentum_commutation_position {d : ℕ} (i j k : Fin d) : {L[i,j], x[k]} =  
(Complex.I * ħ * δ[i,k]) • x[j] - (Complex.I * ħ * δ[j,k]) • x[i]
```

lemma angularMomentum_commutation_radiusRegPow[\[source\]](#)

```
lemma angularMomentum_commutation_radiusRegPow (hε : 0 < ε) (i j : Fin d) :
  {L[i,j], radiusRegPowOperator (d := d) ε s} = 0
```

lemma angularMomentumSqr_commutation_radiusRegPow[\[source\]](#)

```
lemma angularMomentumSqr_commutation_radiusRegPow (hε : 0 < ε) :
  {angularMomentumOperatorSqr (d := d), radiusRegPowOperator (d := d) ε s} = 0
```

lemma angularMomentum_commutation_momentum[\[source\]](#)

```
lemma angularMomentum_commutation_momentum {d : ℕ} (i j k : Fin d) : {L[i,j], p[k]} =
  (Complex.I * ħ * δ[i,k]) • p[j] - (Complex.I * ħ * δ[j,k]) • p[i]
```

lemma momentum_comp_angularMomentum_eq[\[source\]](#)

```
lemma momentum_comp_angularMomentum_eq {d : ℕ} (i j k : Fin d) : p[k] •L L[i,j] =
  L[i,j] •L p[k] - (Complex.I * ħ * δ[i,k]) • p[j] + (Complex.I * ħ * δ[j,k]) • p[i]
```

lemma angularMomentum_commutation_momentumSqr[\[source\]](#)

```
lemma angularMomentum_commutation_momentumSqr {d : ℕ} (i j : Fin d) :
  {L[i,j], momentumOperatorSqr (d := d)} = 0
```

lemma momentumSqr_comp_angularMomentum_commute[\[source\]](#)

```
lemma momentumSqr_comp_angularMomentum_commute {d : ℕ} (i j : Fin d) :
  p2 •L L[i,j] = L[i,j] •L p2
```

lemma angularMomentumSqr_commutation_momentumSqr[\[source\]](#)

```
lemma angularMomentumSqr_commutation_momentumSqr {d : ℕ} :
  {angularMomentumOperatorSqr (d := d), momentumOperatorSqr (d := d)} = 0
```

lemma angularMomentum_commutation_angularMomentum[\[source\]](#)

```
lemma angularMomentum_commutation_angularMomentum {d : ℕ} (i j k l : Fin d) : {L[i,j], L[k,l]} =
  (Complex.I * ħ * δ[i,k]) • L[j,l] - (Complex.I * ħ * δ[i,l]) • L[j,k]
  - (Complex.I * ħ * δ[j,k]) • L[i,l] + (Complex.I * ħ * δ[j,l]) • L[i,k]
```

lemma angularMomentumSqr_commutation_angularMomentum[\[source\]](#)

```
lemma angularMomentumSqr_commutation_angularMomentum {d : ℕ} (i j : Fin d) :
  {angularMomentumOperatorSqr (d := d), L[i,j]} = 0
```

2.32 PhysLean.QuantumMechanics.DDimensions.Operators.Momentum

[PhysLean/QuantumMechanics/DDimensions/Operators/Momentum.lean](#) on GitHub.

def momentumOperator[\[source\]](#)

Component i of the momentum operator is the continuous linear map from $\mathcal{S}(\text{Space } d, \mathbb{C})$ to itself which maps ψ to $-i\hbar \partial_i \psi$.

```
def momentumOperator {d : ℕ} (i : Fin d) :  $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$ 
```

lemma momentumOperator_apply_fun[\[source\]](#)

```
lemma momentumOperator_apply_fun {d : ℕ} (i : Fin d) ( $\psi : \mathcal{S}(\text{Space } d, \mathbb{C})$ ) :  
  p[i]  $\psi = (- \text{Complex.I} * \hbar) \cdot \partial[i] \psi$ 
```

lemma momentumOperator_apply[\[source\]](#)

```
lemma momentumOperator_apply {d : ℕ} (i : Fin d) ( $\psi : \mathcal{S}(\text{Space } d, \mathbb{C})$ ) (x : Space d) :  
  p[i]  $\psi x = - \text{Complex.I} * \hbar * \partial[i] \psi x$ 
```

def momentumOperatorSqr[\[source\]](#)

The square of the momentum operator, $\mathbf{p}^2 = \sum_i \mathbf{p}_i \circ \mathbf{p}_i$.

```
def momentumOperatorSqr {d : ℕ} :  $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$ 
```

lemma momentumOperatorSqr_apply[\[source\]](#)

```
lemma momentumOperatorSqr_apply {d : ℕ} ( $\psi : \mathcal{S}(\text{Space } d, \mathbb{C})$ ) (x : Space d) :  
   $\mathbf{p}^2 \psi x = \sum i, \mathbf{p}[i] (\mathbf{p}[i] \psi) x$ 
```

2.33 PhysLean.QuantumMechanics.DDimensions.Operators.Position

[PhysLean/QuantumMechanics/DDimensions/Operators/Position.lean](#) on GitHub.

def positionOperator[\[source\]](#)

Component i of the position operator is the continuous linear map from $\mathcal{S}(\text{Space } d, \mathbb{C})$ to itself which maps ψ to $x_i \psi$.

```
def positionOperator (i : Fin d) :  $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$ 
```

lemma positionOperator_apply_fun[\[source\]](#)

```
lemma positionOperator_apply_fun (i : Fin d) ( $\psi : \mathcal{S}(\text{Space } d, \mathbb{C})$ ) :  
  x[i]  $\psi = (\text{fun } x \mapsto x i * \psi x)$ 
```

lemma positionOperator_apply[\[source\]](#)

```
lemma positionOperator_apply (i : Fin d) ( $\psi : \mathcal{S}(\text{Space } d, \mathbb{C})$ ) (x : Space d) :  
  x[i]  $\psi x = x i * \psi x$ 
```


def normRegularizedPow[\[source\]](#)

Power of regularized norm, $(\|x\|^2 + \varepsilon^2)^{s/2}$

```
private def normRegularizedPow (ε s : ℝ) : Space d → ℝ
```

lemma normRegularizedPow_hasTemperateGrowth[\[source\]](#)

```
private lemma normRegularizedPow_hasTemperateGrowth (hε : 0 < ε) :
  HasTemperateGrowth (normRegularizedPow (d := d) ε s)
```

def radiusRegPowOperator[\[source\]](#)

The radius operator to power s , regularized by $\varepsilon \neq 0$, is the continuous linear map from $\mathcal{S}(\text{Space } d, \mathbb{C})$ to itself which maps ψ to $(\|x\|^2 + \varepsilon^2)^{s/2} \cdot \psi$.

```
def radiusRegPowOperator (ε s : ℝ) :  $\mathcal{S}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathcal{S}(\text{Space } d, \mathbb{C})$ 
```

lemma radiusRegPowOperator_apply_fun[\[source\]](#)

```
lemma radiusRegPowOperator_apply_fun (hε : 0 < ε) :
  r[ε,s] ψ = fun x ↦ ( $\|x\|^2 + \varepsilon^2$ ) ^ (s / 2) • ψ x
```

lemma radiusRegPowOperator_apply[\[source\]](#)

```
lemma radiusRegPowOperator_apply (hε : 0 < ε) :
  r[ε,s] ψ x = ( $\|x\|^2 + \varepsilon^2$ ) ^ (s / 2) • ψ x
```

lemma radiusRegPowOperator_comp_eq[\[source\]](#)

```
lemma radiusRegPowOperator_comp_eq (hε : 0 < ε) (s t : ℝ) :
  (radiusRegPowOperator (d := d) ε s) ∘L r[ε,t] = r[ε,s+t]
```

lemma radiusRegPowOperator_zero[\[source\]](#)

```
lemma radiusRegPowOperator_zero (hε : 0 < ε) :
  r[ε,0] = ContinuousLinearMap.id  $\mathbb{C} \mathcal{S}(\text{Space } d, \mathbb{C})$ 
```

lemma positionOperatorSqr_eq[\[source\]](#)

```
lemma positionOperatorSqr_eq (hε : 0 < ε) :  $\sum i, x[i] \circL x[i] =$ 
  r[ε,2] - ε ^ 2 • ContinuousLinearMap.id  $\mathbb{C} \mathcal{S}(\text{Space } d, \mathbb{C})$ 
```

2.34 PhysLean.QuantumMechanics.DDimensions.Operators.Unbounded

[PhysLean/QuantumMechanics/DDimensions/Operators/Unbounded.lean](#) on GitHub.

structure UnboundedOperator[\[source\]](#)

An UnboundedOperator is a linear map from a submodule of the Hilbert space to the Hilbert space, assumed to be both densely defined and closable.

```
@[ext]
structure UnboundedOperator
  (HS : Type*) [NormedAddCommGroup HS] [InnerProductSpace ℂ HS] [CompleteSpace HS]
  extends LinearPMap ℂ HS HS where
  dense_domain : Dense (domain : Set HS)
  is_closable : toLinearPMap.IsClosable
```

lemma ext' [\[source\]](#)

```
lemma ext' (U T : UnboundedOperator HS) (h : U.toLinearPMap = T.toLinearPMap) : U = T
```

lemma ext_iff' [\[source\]](#)

```
lemma ext_iff' (U T : UnboundedOperator HS) : U = T ↔ U.toLinearPMap = T.toLinearPMap
```

def ofSymmetric [\[source\]](#)

An UnboundedOperator constructed from a symmetric linear map on a dense submodule E.

```
def ofSymmetric (f : E →ℓ[ℂ] E) (hf : f.IsSymmetric) : UnboundedOperator HS where
```

lemma ofSymmetric_apply [\[source\]](#)

```
@[simp]
lemma ofSymmetric_apply {f : E →ℓ[ℂ] E} {hf : f.IsSymmetric} (ψ : E) :
  (ofSymmetric (hE := hE) f hf).toLinearPMap ψ = E.subtypeL (f ψ)
```

def closure [\[source\]](#)

The closure of an unbounded operator.

```
noncomputable def closure : UnboundedOperator HS where
```

lemma closure_toLinearPMap [\[source\]](#)

```
@[simp]
lemma closure_toLinearPMap : U.closure.toLinearPMap = U.toLinearPMap.closure
```

def IsClosed [\[source\]](#)

An unbounded operator is closed iff the graph of its defining LinearPMap is closed.

```
def IsClosed : Prop
```

lemma closure_isClosed [\[source\]](#)

```
lemma closure_isClosed : U.closure.IsClosed
```

lemma adjoint_isClosable_dense[\[source\]](#)

The adjoint of a densely defined, closable LinearPMap is densely defined.

```
lemma adjoint_isClosable_dense (f : LinearPMap ℂ HS HS) (h_dense : Dense (f.domain : Set HS))
  (h_closable : f.IsClosable) : Dense (f†.domain : Set HS)
```

def adjoint[\[source\]](#)

The adjoint of an unbounded operator, denoted as $U^\dagger$.

```
noncomputable def adjoint : UnboundedOperator HS where
```

instance instStar[\[source\]](#)

```
noncomputable instance instStar : Star (UnboundedOperator HS) where
```

lemma adjoint_toLinearPMap[\[source\]](#)

```
@[simp]
```

```
lemma adjoint_toLinearPMap : U†.toLinearPMap = U.toLinearPMap†
```

lemma isSelfAdjoint_def[\[source\]](#)

```
lemma isSelfAdjoint_def : IsSelfAdjoint U ↔ U† = U
```

lemma isSelfAdjoint_iff[\[source\]](#)

```
lemma isSelfAdjoint_iff : IsSelfAdjoint U ↔ IsSelfAdjoint U.toLinearPMap
```

lemma adjoint_isClosed[\[source\]](#)

```
lemma adjoint_isClosed : (U†).IsClosed
```

lemma closure_adjoint_eq_adjoint[\[source\]](#)

```
lemma closure_adjoint_eq_adjoint : U.closure† = U†
```

lemma adjoint_adjoint_eq_closure[\[source\]](#)

```
lemma adjoint_adjoint_eq_closure : U†† = U.closure
```

def IsGeneralizedEigenvector[\[source\]](#)

A map $F : D(U) \rightarrow L[\mathbb{C}] \mathbb{C}$ is a generalized eigenvector of an unbounded operator U if there is an eigenvalue c such that for all $\psi \in D(U)$, $F(U \psi) = c \cdot F \psi$.

```
def IsGeneralizedEigenvector (F : U.domain → L[ℂ] ℂ) (c : ℂ) : Prop
```

lemma isGeneralizedEigenvector_ofSymmetric_iff[\[source\]](#)

```

lemma isGeneralizedEigenvector_ofSymmetric_iff
  {f : E →L[ $\mathbb{C}$ ] E} (hf : f.IsSymmetric) (F : E →L[ $\mathbb{C}$ ]  $\mathbb{C}$ ) (c :  $\mathbb{C}$ ) :
  IsGeneralizedEigenvector (ofSymmetric (hE := hE) f hf) F c ↔ ∀ ψ : E, F (f ψ) = c • F ψ

```

2.35 PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Basic

[PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Basic.lean](#) on GitHub.

abbrev SpaceDHilbertSpace[\[source\]](#)

The Hilbert space for single-particle quantum mechanics on Space d is defined to be $L^2(\text{Space } d, \mathbb{C})$, the space of almost-everywhere equal equivalence classes of square-integrable functions from Space d to $\mathbb{C}$.

```

abbrev SpaceDHilbertSpace (d :  $\mathbb{N}$ )

```

def toBra[\[source\]](#)

The anti-linear map from the Hilbert space to its dual.

```

def toBra {d :  $\mathbb{N}$ } : SpaceDHilbertSpace d →sl[starRingEnd  $\mathbb{C}$ ] (StrongDual  $\mathbb{C}$  (SpaceDHilbertSpace d))

```

lemma toBra_apply[\[source\]](#)

```

@[simp]
lemma toBra_apply (f g : SpaceDHilbertSpace d) : toBra f g = ⟨⟨f, g⟩⟩_ $\mathbb{C}$ 

```

lemma toBra_surjective[\[source\]](#)

The anti-linear map toBra taking a ket to its corresponding bra is surjective.

```

lemma toBra_surjective : Function.Surjective (toBra (d := d))

```

lemma toBra_injective[\[source\]](#)

The anti-linear map toBra taking a ket to its corresponding bra is injective.

```

lemma toBra_injective : Function.Injective (toBra (d := d))

```

def MemHS[\[source\]](#)

The proposition MemHS f for a function $f : \text{Space } d \rightarrow \mathbb{C}$ is defined to be true if the function f can be lifted to the Hilbert space.

```

def MemHS (f : Space d →  $\mathbb{C}$ ) : Prop

```

lemma aeStronglyMeasurable_of_memHS[\[source\]](#)

```

lemma aeStronglyMeasurable_of_memHS {f : Space d →  $\mathbb{C}$ } (h : MemHS f) : AEStronglyMeasurable f

```

lemma memHS_iff[\[source\]](#)

A function f satisfies $\text{MemHS } f$ if and only if it is almost everywhere strongly measurable and square integrable.

```
lemma memHS_iff {f : Space d → ℂ} : MemHS f ↔
  AESTronglyMeasurable f ∧ Integrable (fun x ↦ ||f x|| ^ 2)
```

lemma zero_memHS[\[source\]](#)

```
@[simp]
```

```
lemma zero_memHS : MemHS (d := d) 0
```

lemma zero_fun_memHS[\[source\]](#)

```
@[simp]
```

```
lemma zero_fun_memHS : MemHS (fun _ : Space d ↦ (0 : ℂ))
```

lemma memHS_add[\[source\]](#)

```
lemma memHS_add {f g : Space d → ℂ} (hf : MemHS f) (hg : MemHS g) : MemHS (f + g)
```

lemma memHS_const_smul[\[source\]](#)

```
lemma memHS_const_smul {f : Space d → ℂ} {c : ℂ} (hf : MemHS f) : MemHS (c • f)
```

lemma memHS_of_ae[\[source\]](#)

```
lemma memHS_of_ae {g : Space d → ℂ} (f : Space d → ℂ) (hf : MemHS f) (hfg : f =[volume] g) :
  MemHS g
```

lemma aeEqFun_mk_mem_iff[\[source\]](#)

```
lemma aeEqFun_mk_mem_iff (f : Space d → ℂ) (hf : AESTronglyMeasurable f volume) :
  AEEqFun.mk f hf ∈ SpaceDHilbertSpace d ↔ MemHS f
```

def mk[\[source\]](#)

Given a function $f : \text{Space } d \rightarrow \mathbb{C}$ such that $\text{MemHS } f$ is true via hf , $\text{SpaceDHilbertSpace.mk } f$ is the element of the Hilbert space defined by f .

```
def mk {f : Space d → ℂ} (hf : MemHS f) : SpaceDHilbertSpace d
```

lemma coe_hilbertSpace_memHS[\[source\]](#)

```
lemma coe_hilbertSpace_memHS (f : SpaceDHilbertSpace d) : MemHS (f : Space d → ℂ)
```

lemma mk_surjective[\[source\]](#)

```
lemma mk_surjective (f : SpaceDHilbertSpace d) :
  ∃ (g : Space d → ℂ) (hg : MemHS g), mk hg = f
```

lemma coe_mk_ae [\[source\]](#)

```
lemma coe_mk_ae {f : Space d → ℂ} (hf : MemHS f) : (mk hf) =ᵐ[volume] f
```

lemma inner_mk_mk [\[source\]](#)

```
lemma inner_mk_mk {f g : Space d → ℂ} (hf : MemHS f) (hg : MemHS g) :
  ⟨mk hf, mk hg⟩_ℂ = ∫ x : Space d, starRingEnd ℂ (f x) * g x
```

lemma eLpNorm_mk [\[source\]](#)

```
@[simp]
lemma eLpNorm_mk {f : Space d → ℂ} {hf : MemHS f} : eLpNorm (mk hf) 2 = eLpNorm f 2
```

lemma mem_iff [\[source\]](#)

```
lemma mem_iff {f : Space d → ℂ} (hf : AESTronglyMeasurable f volume) :
  AEEqFun.mk f hf ∈ SpaceDHilbertSpace d ↔ Integrable (fun x ↦ ||f x|| ^ 2)
```

lemma mk_add [\[source\]](#)

```
@[simp]
lemma mk_add {f g : Space d → ℂ} {hf : MemHS f} {hg : MemHS g} :
  mk (memHS_add hf hg) = mk hf + mk hg
```

lemma mk_const_smul [\[source\]](#)

```
@[simp]
lemma mk_const_smul {f : Space d → ℂ} {c : ℂ} {hf : MemHS f} :
  mk (memHS_const_smul (c := c) hf) = c • mk hf
```

lemma mk_eq_iff [\[source\]](#)

```
lemma mk_eq_iff {f g : Space d → ℂ} {hf : MemHS f} {hg : MemHS g} :
  mk hf = mk hg ↔ f =ᵐ[volume] g
```

lemma ext_iff [\[source\]](#)

```
lemma ext_iff {f g : SpaceDHilbertSpace d} :
  f = g ↔ (f : Space d → ℂ) =ᵐ[volume] (g : Space d → ℂ)
```

2.36 PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmodule

[PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/SchwartzSubmodule.lean](#) on GitHub.

def schwartzIncl [\[source\]](#)

The continuous linear map including Schwartz functions into SpaceDHilbertSpace d.

```
def schwartzIncl {d : ℕ} : S(Space d, ℂ) →L[ℂ] SpaceDHilbertSpace d
```

lemma schwartzIncl_injective[\[source\]](#)

```
lemma schwartzIncl_injective {d : ℕ} : Function.Injective (schwartzIncl (d := d))
```

lemma schwartzIncl_coe_ae[\[source\]](#)

```
lemma schwartzIncl_coe_ae {d : ℕ} (f : S(Space d, ℂ)) : f.1 ==[volume] (schwartzIncl f)
```

lemma schwartzIncl_inner[\[source\]](#)

```
lemma schwartzIncl_inner {d : ℕ} (f g : S(Space d, ℂ)) :
  ⟨⟨schwartzIncl f, schwartzIncl g⟩⟩_ℂ = ∫ x : Space d, starRingEnd ℂ (f x) * g x
```

abbrev schwartzSubmodule[\[source\]](#)

The submodule of SpaceDHilbertSpace d consisting of Schwartz functions.

```
abbrev schwartzSubmodule (d : ℕ)
```

lemma schwartzSubmodule_dense[\[source\]](#)

```
lemma schwartzSubmodule_dense {d : ℕ} :
  Dense (schwartzSubmodule d : Set (SpaceDHilbertSpace d))
```

2.37 PhysLean.Relativity.Tensors.Contraction.Pure

[PhysLean/Relativity/Tensors/Contraction/Pure.lean](#) on GitHub.

lemma fst_ne_dropPairEmb_pre[\[source\]](#)

```
@[simp]
lemma fst_ne_dropPairEmb_pre (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ i = dropPairEmb i j m
```

lemma dropPairEmb_ne_fst[\[source\]](#)

```
@[simp]
lemma dropPairEmb_ne_fst (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ dropPairEmb i j m = i
```

lemma snd_ne_dropPairEmb_pre[\[source\]](#)

```
@[simp]
lemma snd_ne_dropPairEmb_pre (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ j = (dropPairEmb i j) m
```

lemma dropPairEmb_ne_snd[\[source\]](#)

```
@[simp]
lemma dropPairEmb_ne_snd (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ dropPairEmb i j m = j
```

2.38 PhysLean.Relativity.Tensors.Evaluation

[PhysLean/Relativity/Tensors/Evaluation.lean](#) on GitHub.

lemma evalT_pure

[\[source\]](#)

```
@[simp]
lemma evalT_pure {n : ℕ} {c : Fin (n + 1) → C} (i : Fin (n + 1))
  (b : Fin (S.repDim (c i))) (p : Pure S c) :
  evalT i b p.toTensor = Pure.evalP i b p
```

2.39 PhysLean.Relativity.Tensors.RealTensor.ToComplex

[PhysLean/Relativity/Tensors/RealTensor/ToComplex.lean](#) on GitHub.

lemma complexify_apply

[\[source\]](#)

```
@[simp]
lemma ComponentIdx.complexify_apply {n} {c : Fin n → realLorentzTensor.Color}
  (f : ComponentIdx (S := realLorentzTensor) c) (j : Fin n) :
  (ComponentIdx.complexify f) j = Fin.cast (by
    simp only [repDim_eq_one_plus_dim, Nat.reduceAdd, Function.comp_apply]
    generalize c j = cj
    match cj with
    | .up => rfl
    | .down => rfl) (f j)
```

lemma complexify_toFun_apply

[\[source\]](#)

```
@[simp]
lemma ComponentIdx.complexify_toFun_apply {n} {c : Fin n → realLorentzTensor.Color}
  (f : ComponentIdx (S := realLorentzTensor) c) (j : Fin n) :
  (ComponentIdx.complexify.toFun f) j = (ComponentIdx.complexify f) j
```

lemma toComplex_basis

[\[source\]](#)

toComplex sends basis elements to basis elements.

```
@[simp]
lemma toComplex_basis {n} {c : Fin n → realLorentzTensor.Color}
  (i : ComponentIdx (S := realLorentzTensor) c) :
  toComplex (c := c) ((Tensor.basis (S := realLorentzTensor) c) i) =
  (Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c)) i.complexify
```

lemma toComplex_pure_basisVector

[\[source\]](#)

toComplex on a pure basis vector.

```
@[simp]
lemma toComplex_pure_basisVector {n} {c : Fin n → realLorentzTensor.Color}
  (b : ComponentIdx (S := realLorentzTensor) c) :
  toComplex (c := c) (Pure.basisVector c b |>.toTensor)
  =
  (Pure.basisVector (colorToComplex ∘ c) b.complexify).toTensor
```


lemma toComplex_map_smul[\[source\]](#)

```
lemma toComplex_map_smul {n} (c : Fin n → realLorentzTensor.Color) (r : ℝ) (t : ℝT(3, c)) :
  toComplex (c := c) (r • t) = (Complex.ofReal r) • toComplex (c := c) t
```

lemma permCond_prodTColorToComplex[\[source\]](#)

```
lemma permCond_prodTColorToComplex {n m : ℕ}
  {c : Fin n → realLorentzTensor.Color} {c1 : Fin m → realLorentzTensor.Color} :
  PermCond (Fin.append (colorToComplex ∘ c) (colorToComplex ∘ c1))
    (colorToComplex ∘ Fin.append c c1)
    (id : Fin (n + m) → Fin (n + m))
```

lemma cast_componentIdx_apply[\[source\]](#)

```
private lemma cast_componentIdx_apply {n : ℕ} {c c' : Fin n → complexLorentzTensor.Color}
  (h : c' = c) (f : ComponentIdx (S := complexLorentzTensor) c') (x : Fin n) :
  (cast (congr_arg ComponentIdx h) f) x =
    Fin.cast (congr_arg (fun c => complexLorentzTensor.repDim (c x)) h) (f x)
```

lemma cast_componentIdx_eq_fun[\[source\]](#)

```
@[simp]
private lemma cast_componentIdx_eq_fun {n : ℕ}
  {c c' : Fin n → complexLorentzTensor.Color}
  (h : c' = c) (f : ComponentIdx (S := complexLorentzTensor) c') :
  cast (congr_arg ComponentIdx h) f =
    (fun x =>
      Fin.cast (congr_arg (fun col => complexLorentzTensor.repDim (col x)) h) (f x))
```

lemma complexify_prod[\[source\]](#)

complexify commutes with prod of component indices.

```
@[simp]
lemma complexify_prod {n m : ℕ}
  {c : Fin n → realLorentzTensor.Color} {c1 : Fin m → realLorentzTensor.Color}
  (b : ComponentIdx (S := realLorentzTensor) c)
  (b1 : ComponentIdx (S := realLorentzTensor) c1) :
  ComponentIdx.complexify (c := Fin.append c c1) (b.prod b1)
  =
  cast (congr_arg ComponentIdx (colorToComplex_append c c1).symm)
    ((ComponentIdx.complexify (c := c) b).prod (ComponentIdx.complexify (c := c1) b1))
```

lemma complexify_comp_dropPairEmb[\[source\]](#)

complexify commutes with precomposition by dropPairEmb. We use fun k => b (Pure.dropPairEmb i j k) and direct application (ComponentIdx.complexify b) (Pure.dropPairEmb i j m) rather than composition so that dependent ComponentIdx types unify correctly (avoiding Function.comp type mismatch).

```
@[simp]
lemma ComponentIdx.complexify_comp_dropPairEmb
  {n : ℕ} {c : Fin (n + 1 + 1) → realLorentzTensor.Color}
  {i j : Fin (n + 1 + 1)} (b : ComponentIdx (S := realLorentzTensor) c) (m : Fin n) :
```

```
(ComponentIdx.complexify (c := c ◦ Pure.dropPairEmb i j)
  (fun k => b (Pure.dropPairEmb i j k))) m =
(ComponentIdx.complexify (c := c) b) (Pure.dropPairEmb i j m)
```

lemma toComplex_contrP_basisVector[\[source\]](#)

For a real basis vector, toComplex(contrP(basisVector c b)) equals contrP(basisVector (colorToComplex ◦ c) (complexify b)) (complex species).

```
lemma toComplex_contrP_basisVector {n : ℕ} {c : Fin (n + 1 + 1) → realLorentzTensor.Color}
  {i j : Fin (n + 1 + 1)} (h : i ≠ j ∧ (realLorentzTensor).τ (c i) = c j)
  (b : ComponentIdx (S := realLorentzTensor) c) :
toComplex (c := c ◦ Pure.dropPairEmb i j)
  (Pure.contrP (S := realLorentzTensor) i j h (Pure.basisVector c b))
=
Pure.contrP (S := complexLorentzTensor) i j
  (by
    simp [Function.comp_apply] using And.intro h.1
    (by simp [tau_colorToComplex] using congrArg colorToComplex h.2))
  (Pure.basisVector (colorToComplex ◦ c) (ComponentIdx.complexify b))
```

lemma complexify_comp_succAbove[\[source\]](#)

complexify commutes with precomposition by succAbove.

```
@[simp]
lemma ComponentIdx.complexify_comp_succAbove
  {n : ℕ} {c : Fin (n + 1) → realLorentzTensor.Color} (i : Fin (n + 1))
  (b : ComponentIdx (S := realLorentzTensor) c) (m : Fin n) :
(ComponentIdx.complexify (c := c ◦ i.succAbove) (fun k => b (i.succAbove k))) m =
(ComponentIdx.complexify (c := c) b) (i.succAbove m)
```

lemma toComplex_evalP_basisVector[\[source\]](#)

For a real basis vector, toComplex(evalP(basisVector c b)) equals evalP(basisVector (colorToComplex ◦ c) (complexify b)) (complex species).

```
lemma toComplex_evalP_basisVector {n : ℕ} {c : Fin (n + 1) → realLorentzTensor.Color}
  (i : Fin (n + 1)) (b : Fin ((realLorentzTensor).repDim (c i)))
  (b' : ComponentIdx (S := realLorentzTensor) c) :
toComplex (c := c ◦ i.succAbove)
  (Pure.evalP (S := realLorentzTensor) i b (Pure.basisVector c b'))
=
permT (S := complexLorentzTensor) (σ := (id : Fin n → Fin n))
  (by simp [Function.comp_apply])
  (Pure.evalP (S := complexLorentzTensor) i (evalIdxToComplex (c := c) i b)
    (Pure.basisVector (colorToComplex ◦ c) (ComponentIdx.complexify b')))
```

2.40 PhysLean.Relativity.Tensors.RealTensor.Velocity.Basic

[PhysLean/Relativity/Tensors/RealTensor/Velocity/Basic.lean](#) on GitHub.

lemma one_add_minkowskiProduct_ne_zero[\[source\]](#)

```
lemma one_add_minkowskiProduct_ne_zero (u v : Velocity d) :
```

$$1 + \langle\langle u.1, v.1 \rangle\rangle_m \neq 0$$

2.41 PhysLean.Relativity.Tensors.TensorSpecies.Basic

[PhysLean/Relativity/Tensors/TensorSpecies/Basic.lean](#) on GitHub.

lemma castToField_eq_self

[\[source\]](#)

```
lemma castToField_eq_self {S : TensorSpecies k C G} {c}
  (v : (↑((⊔_ (Discrete C ↪ Rep k G)).obj { as := c })).V)) :
  S.castToField v = v
```

2.42 PhysLean.SpaceAndTime.Space.Basic

[PhysLean/SpaceAndTime/Space/Basic.lean](#) on GitHub.

lemma volume_closedBall_ne_zero

[\[source\]](#)

```
lemma volume_closedBall_ne_zero {d : ℕ} (x : Space d.succ) (r : ℝ) (hr : 0 < r) :
  volume (Metric.closedBall x r) ≠ 0
```

lemma volume_closedBall_ne_top

[\[source\]](#)

```
lemma volume_closedBall_ne_top {d : ℕ} (x : Space d.succ) (r : ℝ) :
  volume (Metric.closedBall x r) ≠ ⊤
```

2.43 PhysLean.SpaceAndTime.Space.Derivatives.Basic

[PhysLean/SpaceAndTime/Space/Derivatives/Basic.lean](#) on GitHub.

lemma deriv_const_smul

[\[source\]](#)

Space derivatives on scalar times function.

```
lemma deriv_const_smul [NormedAddCommGroup M] [NormedSpace ℝ M] [Semiring R]
  [Module R M] [SMulCommClass ℝ R M] [ContinuousConstSMul R M] {f : Space d → M} (c : R)
  (h : Differentiable ℝ f) : ∂[u] (c • f) = c • ∂[u] f
```

2.44 PhysLean.SpaceAndTime.Space.LengthUnit

[PhysLean/SpaceAndTime/Space/LengthUnit.lean](#) on GitHub.

lemma val_ne_zero

[\[source\]](#)

```
@[simp]
lemma val_ne_zero (x : LengthUnit) : x.val ≠ 0
```

lemma div_ne_zero

[\[source\]](#)

```
@[simp]
lemma div_ne_zero (x y : LengthUnit) : ¬ x / y = (0 : ℝ≥0)
```

2.45 PhysLean.SpaceAndTime.Time.TimeUnit

[PhysLean/SpaceAndTime/Time/TimeUnit.lean](#) on GitHub.

lemma val_ne_zero

[\[source\]](#)

```
@[simp]
lemma val_ne_zero (x : TimeUnit) : x.val ≠ 0
```

lemma div_ne_zero

[\[source\]](#)

```
@[simp]
lemma div_ne_zero (x y : TimeUnit) : ¬ x / y = (0 : ℝ≥0)
```

2.46 PhysLean.StatisticalMechanics.BoltzmannConstant

[PhysLean/StatisticalMechanics/BoltzmannConstant.lean](#) on GitHub.

lemma kB_ne_zero

[\[source\]](#)

The Boltzmann constant is not equal to zero.

```
lemma kB_ne_zero : kB ≠ 0
```

2.47 PhysLean.Thermodynamics.Temperature.TemperatureUnits

[PhysLean/Thermodynamics/Temperature/TemperatureUnits.lean](#) on GitHub.

lemma val_ne_zero

[\[source\]](#)

```
@[simp]
lemma val_ne_zero (x : TemperatureUnit) : x.val ≠ 0
```

lemma div_ne_zero

[\[source\]](#)

```
@[simp]
lemma div_ne_zero (x y : TemperatureUnit) : ¬ x / y = (0 : ℝ≥0)
```

2.48 PhysLean.Units.Basic

[PhysLean/Units/Basic.lean](#) on GitHub.

lemma dimScale_ne_zero

[\[source\]](#)

```
@[simp]
lemma dimScale_ne_zero (u1 u2 : UnitChoices) (d : Dimension) :
  dimScale u1 u2 d ≠ 0
```